

LAPORAN TUGAS 7

SISTEM OPERASI



Dosen Pengampu:

Triawan Adi Cahyanto, M.Kom

Disusun Oleh:

Mochamad Rio Aprilian (2410651040)

PRODI TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH JEMBER

2025

DAFTAR ISI

BAB 1	3
PENDAHULUAN	3
1.1 LATAR BELAKANG	3
1.2 TUJUAN PRAKTIKUM	3
1.3 SPESIFIKASI SISTEM.....	3
BAB II	4
DASAR TEORI.....	4
2.1 MEMORY MANAGEMENT.....	4
2.2 CONTIGUOUS ALLOCATION.....	4
2.3 PAGING.....	4
2.4 PAGE TABLE STRUCTURES.....	4
2.5 SWAPPING	5
2.6 ARCHITECTURE	5
BAB III.....	6
JAWABAN TUGAS PRAKTIKUM	6
3.1 TOPIK 1.....	6
3.2 TOPIK 2.....	11
3.3 TOPIK 3.....	19
3.4 TOPIK 4.....	41
3.5 TOPIK 5.....	48
3.6 TOPIK 6.....	55
3.7 TOPIK 7.....	Error! Bookmark not defined.
BAB IV	60
ANALISIS DAN PEMBAHASAN.....	60
4.1 ANALISIS TUGAS 1	60
4.2 ANALISIS TUGAS 2	62
4.3 ANALISIS TUGAS 3	63
4.4 ANALISIS TUGAS 4	67
4.5 ANALISIS TUGAS 5	70
4.6 ANALISIS TUGAS 6	71
BAB V	73
KESIMPULAN DAN SARAN	73
DAFTAR PUSTAKA	74

BAB 1

PENDAHULUAN

1.1 LATAR BELAKANG

Praktikum Sistem Operasi ini dirancang untuk memberikan pemahaman komprehensif mengenai mekanisme manajemen memori melalui implementasi dan analisis langsung menggunakan bahasa C dan perintah sistem Linux. Kode-kode yang dihasilkan mencakup eksplorasi struktur memori proses, simulasi algoritma alokasi memori, paging, page replacement, struktur page table, swapping, hingga pengamatan arsitektur memori nyata. Setiap program tidak hanya menampilkan hasil eksekusi, tetapi juga merepresentasikan konsep teoretis seperti segmentasi memori, fragmentasi, page fault, dan page replacement secara konkret. Dengan mengamati alamat memori, alur alokasi dan dealokasi, serta statistik performa, mahasiswa dapat memahami hubungan erat antara teori sistem operasi dan implementasi nyata pada level perangkat lunak dan perangkat keras. Pendekatan berbasis eksperimen ini diharapkan mampu meningkatkan kemampuan analisis kritis terhadap kinerja dan efisiensi manajemen memori dalam sistem operasi modern.

1.2 TUJUAN PRAKTIKUM

- Memahami konsep manajemen memori sistem operasi melalui implementasi langsung program simulasi dan observasi memory layout proses.
- Menganalisis dan membandingkan kinerja berbagai algoritma alokasi memori, paging, dan swapping berdasarkan parameter performa yang terukur.
- Mengaitkan teori arsitektur memori dengan implementasi nyata pada sistem Linux menggunakan perintah dan utilitas sistem.

1.3 SPESIFIKASI SISTEM

Sistem Operasi: Linux (Ubuntu/Debian-based), kernel mendukung ASLR dan virtual memory.

Perangkat Lunak: GCC Compiler, GNU C Library (glibc), Bash Shell, utilitas Linux (lscpu, vmstat, cpuid).

Perangkat Keras: Prosesor 64-bit, RAM minimal 4 GB, mendukung multi-level cache dan paging hardware.

BAB II

DASAR TEORI

2.1 MEMORY MANAGEMENT

Memory management merupakan salah satu fungsi utama sistem operasi yang bertugas mengatur penggunaan memori utama agar dapat dimanfaatkan secara efisien oleh berbagai proses yang berjalan. Sistem operasi harus memastikan setiap proses memperoleh ruang memori yang cukup tanpa saling mengganggu. Selain itu, memory management juga bertanggung jawab terhadap alokasi dan dealokasi memori, proteksi memori, serta pengelolaan memori virtual. Dengan manajemen memori yang baik, sistem dapat meningkatkan kinerja, mencegah konflik antar proses, dan memaksimalkan pemanfaatan sumber daya perangkat keras yang tersedia.

2.2 CONTIGUOUS ALLOCATION

Contiguous allocation adalah metode manajemen memori di mana setiap proses dialokasikan pada satu blok memori yang berurutan atau berdekatan. Metode ini sederhana dan mudah diimplementasikan karena alamat memori dapat dihitung secara langsung. Namun, contiguous allocation memiliki kelemahan utama yaitu terjadinya fragmentasi eksternal, yaitu kondisi ketika terdapat ruang memori kosong tetapi tidak cukup besar untuk menampung proses baru. Untuk mengatasi masalah ini, diperlukan teknik tambahan seperti compaction, yang dapat menambah kompleksitas dan menurunkan performa sistem.

2.3 PAGING

Paging merupakan teknik manajemen memori yang membagi memori fisik menjadi blok-blok kecil berukuran tetap yang disebut frame, dan memori logis proses menjadi page dengan ukuran yang sama. Dengan paging, proses tidak perlu ditempatkan secara berurutan dalam memori fisik, sehingga fragmentasi eksternal dapat dihindari. Sistem operasi menggunakan page table untuk memetakan page ke frame. Meskipun paging meningkatkan fleksibilitas dan efisiensi penggunaan memori, teknik ini dapat menimbulkan overhead tambahan akibat proses translasi alamat memori.

2.4 PAGE TABLE STRUCTURES

Page table structures adalah struktur data yang digunakan sistem operasi untuk menyimpan pemetaan antara page pada memori logis dan frame pada memori fisik. Karena ukuran page table dapat menjadi sangat besar, terutama pada sistem dengan ruang alamat besar, dikembangkan berbagai struktur seperti multilevel page table, hashed page table, dan inverted page table. Tujuan

utama dari struktur ini adalah mengurangi penggunaan memori dan meningkatkan efisiensi akses. Desain page table yang baik sangat berpengaruh terhadap kecepatan translasi alamat dan kinerja sistem secara keseluruhan.

2.5 SWAPPING

Swapping adalah teknik manajemen memori di mana proses yang sedang tidak aktif sementara dipindahkan dari memori utama ke media penyimpanan sekunder seperti hard disk. Ketika proses tersebut dibutuhkan kembali, sistem operasi akan memuatnya kembali ke memori utama. Teknik ini memungkinkan sistem menjalankan lebih banyak proses daripada kapasitas memori fisik yang tersedia. Namun, swapping memiliki kelemahan berupa waktu akses yang lebih lambat karena melibatkan perangkat penyimpanan sekunder, sehingga harus digunakan secara hati-hati agar tidak menurunkan kinerja sistem.

2.6 ARCHITECTURE

Architecture dalam konteks sistem operasi dan manajemen memori merujuk pada desain dan struktur perangkat keras serta cara sistem operasi berinteraksi dengannya. Arsitektur komputer menentukan bagaimana memori diakses, ukuran alamat, serta mekanisme proteksi dan translasi alamat. Perbedaan arsitektur, seperti 32-bit dan 64-bit, sangat mempengaruhi kapasitas memori yang dapat digunakan sistem. Pemahaman terhadap arsitektur sangat penting agar sistem operasi dapat mengelola memori secara optimal dan memanfaatkan fitur perangkat keras secara maksimal.

BAB III

JAWABAN TUGAS PRAKTIKUM

3.1 TUGAS 1

3.1.1 Deskripsi Tugas

Tugas 1 berfokus pada analisis memory layout pada program C dengan memodifikasi file `address_demo.c` agar menampilkan struktur memori secara lebih detail. Mahasiswa diminta menambahkan alokasi array statis dan dinamis serta menampilkan alamat memorinya, membuat beberapa fungsi untuk membandingkan alamat pada text segment, dan mengimplementasikan fungsi rekursif guna mengamati perubahan alamat stack di setiap level pemanggilan. Selain itu, alamat string literal juga harus ditampilkan. Setelah program dijalankan, mahasiswa menganalisis jarak antar segmen memori, kestabilan alamat pada beberapa eksekusi, pengaruh ASLR, serta mencocokkan hasil dengan memory map pada `/proc/[pid]/maps` beserta permission setiap segmen.

3.1.2 Source Code

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

/* prototype */
int main();

/* variabel global */
int global_var = 100;
int static_array[3] = {10, 20, 30};
const char *text = "Main Memory Praktikum";

/* fungsi biasa */
void func1() {}
void func2() {}
void func3() {}

/* fungsi rekursif */
void recursive(int level) {
    int x = level;
    printf(" Level %d local_var address: %p value: %d\n",
        level, (void*)&x, x);

    if (level < 2)
        recursive(level + 1);
}
```

```

void print_memory() {
    int local = 5;
    int *heap1 = malloc(sizeof(int));
    int *heap2 = malloc(sizeof(int));
    int *dyn_array = malloc(3 * sizeof(int));

    /* isi nilai */
    *heap1 = 50;
    *heap2 = 75;
    dyn_array[0] = 1;

    printf("\n=== ENHANCED MEMORY LAYOUT ===\n\n");

    printf("1. Text Segment:\n");
    printf("  main() : %p\n", (void*)main);
    printf("  func1() : %p\n", (void*)func1);
    printf("  func2() : %p\n", (void*)func2);
    printf("  func3() : %p\n\n", (void*)func3);

    printf("2. Data Segment:\n");
    printf("  global_var address: %p value: %d\n",
        (void*)&global_var, global_var);
    printf("  static_array[0] address: %p value: %d\n",
        (void*)&static_array[0], static_array[0]);
    printf("  string literal address: %p value: %s\n\n",
        (void*)text, text);

    printf("3. Heap:\n");
    printf("  heap1 address: %p value: %d\n",
        (void*)heap1, *heap1);
    printf("  heap2 address: %p value: %d\n",
        (void*)heap2, *heap2);
    printf("  dyn_array[0] address: %p value: %d\n\n",
        (void*)&dyn_array[0], dyn_array[0]);

    printf("4. Stack:\n");
    printf("  local address: %p value: %d\n\n",
        (void*)&local, local);

    printf("4. Stack (Recursive Calls):\n");
    recursive(0);

    free(heap1);
    free(heap2);
    free(dyn_array);
}

int main() {
    printf("PID: %d\n", getpid());
    print_memory();
}

```

```
printf("\nTekan Enter untuk melihat memory map...");
getchar();

char cmd[100];
sprintf(cmd, "cat /proc/%d/maps", getpid());
system(cmd);

return 0;
}
```

3.1.3 Screenshot Output

```
rio_regex@DESKTOP-NVPM5HQ:~/praktikum-memory/background$ ./address_demo
PID: 461

=== ENHANCED MEMORY LAYOUT ===

1. Text Segment:
  main() : 0x6033960bb54b
  func1() : 0x6033960bb249
  func2() : 0x6033960bb254
  func3() : 0x6033960bb25f

2. Data Segment:
  global_var address: 0x6033960be010 value: 100
  static_array[0] address: 0x6033960be018 value: 10
  string literal address: 0x6033960bc008 value: Main Memory Praktikum

3. Heap:
  heap1 address: 0x6033988386b0 value: 50
  heap2 address: 0x6033988386d0 value: 75
  dyn_array[0] address: 0x6033988386f0 value: 1

4. Stack:
  local address: 0x7fff9ed66b8c value: 5

4. Stack (Recursive Calls):
  Level 0 local_var address: 0x7fff9ed66b64 value: 0
  Level 1 local_var address: 0x7fff9ed66b34 value: 1
  Level 2 local_var address: 0x7fff9ed66b04 value: 2

Tekan Enter untuk melihat memory map...
6033960ba000-6033960bb000 r--p 00000000 00:30 60084 /home/rio_regex/praktikum-memory/background/address_demo
6033960bb000-6033960bc000 r-xp 00001000 00:30 60084 /home/rio_regex/praktikum-memory/background/address_demo
6033960bc000-6033960bd000 r--p 00002000 00:30 60084 /home/rio_regex/praktikum-memory/background/address_demo
6033960bd000-6033960be000 r--p 00002000 00:30 60084 /home/rio_regex/praktikum-memory/background/address_demo
6033960be000-6033960bf000 rw-p 00003000 00:30 60084 /home/rio_regex/praktikum-memory/background/address_demo
603398838000-6033988386b0 rw-p 00000000 00:00 0 [heap]
796372200000-796372228000 r--p 00000000 00:30 46704 /usr/lib/x86_64-linux-gnu/libc.so.6
796372228000-79637223b000 r-xp 00028000 00:30 46704 /usr/lib/x86_64-linux-gnu/libc.so.6
7963723b0000-7963723ff000 r--p 001b0000 00:30 46704 /usr/lib/x86_64-linux-gnu/libc.so.6
7963723ff000-796372403000 r--p 001fe000 00:30 46704 /usr/lib/x86_64-linux-gnu/libc.so.6
796372403000-796372405000 rw-p 00202000 00:30 46704 /usr/lib/x86_64-linux-gnu/libc.so.6
796372405000-796372412000 rw-p 00000000 00:00 0
796372557000-79637255a000 rw-p 00000000 00:00 0
79637255f000-796372561000 rw-p 00000000 00:00 0
796372561000-796372562000 r--p 00000000 00:30 46701 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
796372562000-79637256d000 r-xp 00001000 00:30 46701 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
79637256d000-796372597000 r--p 0002c000 00:30 46701 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
796372597000-796372599000 r--p 00036000 00:30 46701 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
796372599000-79637259b000 rw-p 00038000 00:30 46701 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7fff9ed48000-7fff9ed69000 rw-p 00000000 00:00 0 [stack]
7fff9ed84000-7fff9ed88000 r--p 00000000 00:00 0 [vvar]
7fff9ed88000-7fff9ed8a000 r-xp 00000000 00:00 0 [vdso]
rio_regex@DESKTOP-NVPM5HQ:~/praktikum-memory/background$ |
```

3.1.4 Penjelasan Implementasi

A. Proses Kompilasi

1. Preprocessing

- Directive diproses:
 - #include <stdio.h>
 - #include <stdlib.h>
 - #include <unistd.h>
- Seluruh deklarasi fungsi dan variabel dibaca oleh preprocessor.
- Komentar dihapus, tidak ada makro yang diekspansi pada kode ini.

2. Compilation

- Kode C diterjemahkan menjadi bahasa assembly.
- Compiler mengklasifikasikan objek program:
 - Fungsi main, func1, func2, func3, recursive, print_memory → **Text Segment**
 - global_var dan static_array → **Data Segment**
 - String literal "Main Memory Praktikum" → **Read-Only Data Segment (.rodata)**
 - Pointer text → **Data Segment**
- Pemeriksaan sintaks dan tipe data dilakukan.

3. Assembly

- Kode assembly diubah menjadi object file (.o).
- Instruksi mesin dibuat untuk:
 - Pemanggilan fungsi
 - Alokasi heap (malloc)
 - Rekursi
 - Operasi I/O (printf, getpid)

4. Linking

- Object file digabung dengan library standar C (glibc).
- Simbol eksternal seperti printf(), malloc(), free(), getpid(), system() dihubungkan.
- Dihasilkan file executable.

B. Proses Eksekusi

1. Pembuatan Proses

- Program dijalankan dan sistem operasi membuat proses baru.
- Proses diberi **PID unik**.
- Segmentasi memori proses disiapkan:

- Text
- Data / BSS / RO Data
- Heap
- Stack

2. Eksekusi Fungsi main()

- Program mulai dari fungsi main.
- PID proses ditampilkan menggunakan getpid().

3. Pemanggilan print_memory()

- Stack frame baru dibuat.
- Variabel lokal local dialokasikan di **Stack**.
- Pemanggilan malloc():
 - heap1, heap2, dan dyn_array dialokasikan di **Heap**.

4. Pengisian dan Akses Data

- Variabel heap diisi nilai (50, 75, 1).
- Variabel global dan static diakses dari **Data Segment**.
- String literal diakses dari **Read-Only Data Segment**.

5. Penampilan Text Segment

- Alamat fungsi (main, func1, func2, func3) dicetak.
- Menunjukkan bahwa fungsi berada di **Text Segment**.

6. Penampilan Data Segment

- Alamat dan nilai:
 - global_var
 - static_array
 - Pointer string literal text
- Menunjukkan pemisahan data terinisialisasi dan read-only.

7. Penampilan Heap

- Alamat memori dinamis (heap1, heap2, dyn_array) dicetak.
- Menunjukkan alokasi dinamis runtime.

8. Penampilan Stack

- Alamat variabel lokal local dicetak.
- Menunjukkan stack frame aktif.

9. Rekursi dan Stack Frame Bertingkat

- Fungsi recursive() dipanggil berulang.
- Setiap level rekursi:
 - Membuat stack frame baru.
 - Alamat variabel lokal berbeda.
- Menunjukkan pertumbuhan stack ke arah alamat lebih kecil.

10. Dealokasi Heap

- free() dipanggil untuk setiap alokasi heap.
- Menghindari memory leak.

11. Menampilkan Memory Map

- Program menunggu input (getchar()).
- Perintah cat /proc/<PID>/maps dijalankan.
- Kernel Linux menampilkan peta memori lengkap proses.

12. Terminasi Program

- main() selesai.
- Sistem operasi menghapus proses dan membersihkan semua resource.

3.2 TUGAS 2

3.2.1 Deskripsi Tugas

Tugas 2 bertujuan untuk mengembangkan program *memory allocator* dengan menambahkan algoritma Next-Fit serta melakukan analisis performa berbagai metode alokasi memori. Mahasiswa diminta mengimplementasikan algoritma Next-Fit yang bekerja seperti First-Fit namun memulai pencarian dari posisi alokasi terakhir. Selain itu, ditambahkan fitur untuk menghitung fragmentasi memori eksternal. Program kemudian diuji menggunakan skenario benchmark untuk membandingkan First-Fit, Best-Fit, Worst-Fit, dan Next-Fit berdasarkan waktu eksekusi, tingkat fragmentasi, dan kegagalan alokasi. Hasil pengujian dianalisis dalam bentuk tabel dan grafik untuk menentukan algoritma yang paling cepat dan paling efisien dalam kondisi tertentu.

3.2.2 Source Code

```
#include <stdio.h>
#include <time.h>

#define MAX_BLOCKS 100
```

```

typedef struct {
    int size;
    int free;    // 1 = free, 0 = allocated
    int process_id;
} Block;

Block memory[MAX_BLOCKS];
int block_count = 1;
int last_alloc_index = 0;

/* ===== INIT MEMORY ===== */
void init_memory(int total_size) {
    memory[0].size = total_size;
    memory[0].free = 1;
    memory[0].process_id = -1;
    block_count = 1;
    last_alloc_index = 0;
}

/* ===== DISPLAY ===== */
void display_memory() {
    printf("\nMemory State:\n");
    for (int i = 0; i < block_count; i++) {
        if (memory[i].free)
            printf("Block %d: FREE | Size = %d\n", i, memory[i].size);
        else
            printf("Block %d: P%d | Size = %d\n", i, memory[i].process_id, memory[i].size);
    }
}

/* ===== DEALLOCATE ===== */
void deallocate(int pid) {
    for (int i = 0; i < block_count; i++) {
        if (!memory[i].free && memory[i].process_id == pid) {
            memory[i].free = 1;
            memory[i].process_id = -1;
            return;
        }
    }
}

/* ===== FIRST FIT ===== */
int first_fit(int pid, int size) {
    for (int i = 0; i < block_count; i++) {
        if (memory[i].free && memory[i].size >= size) {

            if (memory[i].size > size) {
                for (int j = block_count; j > i + 1; j--)
                    memory[j] = memory[j - 1];

                memory[i + 1].size = memory[i].size - size;
            }
        }
    }
}

```

```

        memory[i + 1].free = 1;
        memory[i + 1].process_id = -1;
        block_count++;
    }

    memory[i].size = size;
    memory[i].free = 0;
    memory[i].process_id = pid;
    return 1;
}
}
return 0;
}

/* ===== BEST FIT ===== */
int best_fit(int pid, int size) {
    int best = -1;

    for (int i = 0; i < block_count; i++) {
        if (memory[i].free && memory[i].size >= size) {
            if (best == -1 || memory[i].size < memory[best].size)
                best = i;
        }
    }

    if (best == -1) return 0;

    if (memory[best].size > size) {
        for (int j = block_count; j > best + 1; j--)
            memory[j] = memory[j - 1];

        memory[best + 1].size = memory[best].size - size;
        memory[best + 1].free = 1;
        memory[best + 1].process_id = -1;
        block_count++;
    }

    memory[best].size = size;
    memory[best].free = 0;
    memory[best].process_id = pid;
    return 1;
}

/* ===== WORST FIT ===== */
int worst_fit(int pid, int size) {
    int worst = -1;

    for (int i = 0; i < block_count; i++) {
        if (memory[i].free && memory[i].size >= size) {
            if (worst == -1 || memory[i].size > memory[worst].size)
                worst = i;
        }
    }

```

```

    }
}

if (worst == -1) return 0;

if (memory[worst].size > size) {
    for (int j = block_count; j > worst + 1; j--)
        memory[j] = memory[j - 1];

    memory[worst + 1].size = memory[worst].size - size;
    memory[worst + 1].free = 1;
    memory[worst + 1].process_id = -1;
    block_count++;
}

memory[worst].size = size;
memory[worst].free = 0;
memory[worst].process_id = pid;
return 1;
}

/* ===== NEXT FIT ===== */
int next_fit(int pid, int size) {
    int i = last_alloc_index;
    int scanned = 0;

    while (scanned < block_count) {
        if (memory[i].free && memory[i].size >= size) {

            if (memory[i].size > size) {
                for (int j = block_count; j > i + 1; j--)
                    memory[j] = memory[j - 1];

                memory[i + 1].size = memory[i].size - size;
                memory[i + 1].free = 1;
                memory[i + 1].process_id = -1;
                block_count++;
            }

            memory[i].size = size;
            memory[i].free = 0;
            memory[i].process_id = pid;

            last_alloc_index = i;
            return 1;
        }

        i = (i + 1) % block_count;
        scanned++;
    }

    return 0;
}

```

```

}

/* ===== FRAGMENTATION ===== */
void calculate_fragmentation() {
    int total_free = 0, largest = 0, holes = 0;

    for (int i = 0; i < block_count; i++) {
        if (memory[i].free) {
            total_free += memory[i].size;
            holes++;
            if (memory[i].size > largest)
                largest = memory[i].size;
        }
    }

    float ext = (total_free > 0) ?
        (1.0 - (float)largest / total_free) * 100 : 0;

    printf("Total Free Space    : %d\n", total_free);
    printf("Largest Free Block    : %d\n", largest);
    printf("Free Holes              : %d\n", holes);
    printf("External Fragmentation: %.2f%%\n", ext);
}

/* ===== BENCHMARK ===== */
void benchmark(const char *name, int (*alloc)(int,int)) {
    init_memory(1000);
    clock_t start = clock();

    alloc(1,100);
    alloc(2,200);
    alloc(3,300);
    alloc(4,250);
    alloc(5,150);

    deallocate(2);
    deallocate(4);

    alloc(6,180);
    alloc(7,220);
    alloc(8,100);

    clock_t end = clock();

    printf("\n=== %s ===\n", name);
    printf("Execution Time: %.2f us\n",
        (double)(end-start)*1e6/CLOCKS_PER_SEC);

    calculate_fragmentation();
    display_memory();
}

```

```

/* ===== MAIN ===== */
int main() {
    benchmark("First Fit", first_fit);
    benchmark("Best Fit", best_fit);
    benchmark("Worst Fit", worst_fit);
    benchmark("Next Fit", next_fit);
    return 0;
}

```

3.2.3 Screenshot Output

```

pio_regex@DESKTOP-NVPM5HQ:~/praktikum-memory/contiguous-allocation$ ./memory_allocator

=== First Fit ===
Execution Time: 2.00 us
Total Free Space      : 50
Largest Free Block    : 30
Free Holes            : 2
External Fragmentation: 40.00%

Memory State:
Block 0: P1 | Size = 100
Block 1: P6 | Size = 180
Block 2: FREE | Size = 20
Block 3: P3 | Size = 300
Block 4: P7 | Size = 220
Block 5: FREE | Size = 30
Block 6: P5 | Size = 150

=== Best Fit ===
Execution Time: 2.00 us
Total Free Space      : 50
Largest Free Block    : 30
Free Holes            : 2
External Fragmentation: 40.00%

Memory State:
Block 0: P1 | Size = 100
Block 1: P6 | Size = 180
Block 2: FREE | Size = 20
Block 3: P3 | Size = 300
Block 4: P7 | Size = 220
Block 5: FREE | Size = 30
Block 6: P5 | Size = 150

=== Worst Fit ===
Execution Time: 2.00 us
Total Free Space      : 170
Largest Free Block    : 100
Free Holes            : 2
External Fragmentation: 41.18%

Memory State:
Block 0: P1 | Size = 100
Block 1: P8 | Size = 100
Block 2: FREE | Size = 100
Block 3: P3 | Size = 300
Block 4: P6 | Size = 180
Block 5: FREE | Size = 70
Block 6: P5 | Size = 150

=== Next Fit ===
Execution Time: 2.00 us
Total Free Space      : 50
Largest Free Block    : 30
Free Holes            : 2
External Fragmentation: 40.00%

Memory State:
Block 0: P1 | Size = 100
Block 1: P6 | Size = 180
Block 2: FREE | Size = 20
Block 3: P3 | Size = 300
Block 4: P7 | Size = 220
Block 5: FREE | Size = 30
Block 6: P5 | Size = 150
pio_regex@DESKTOP-NVPM5HQ:~/praktikum-memory/contiguous-allocation$

```

3.2.4 Penjelasan Implementasi

A. Proses Kompilasi

1. Preprocessing

- Preprocessor memproses:
 - `#include <stdio.h>` → fungsi I/O (`printf`)
 - `#include <time.h>` → fungsi waktu (`clock`)
- Makro `MAX_BLOCKS` diganti dengan nilai 100.
- Komentar dihapus.
- Struktur Block didefinisikan untuk merepresentasikan blok memori.

2. Compilation

- Kode C diterjemahkan menjadi **assembly**.
- Compiler menentukan segmentasi data:
 - `memory[MAX_BLOCKS]` → **Data/BSS Segment** (array global)
 - `block_count`, `last_alloc_index` → **Data Segment**
- Semua fungsi (`first_fit`, `best_fit`, `benchmark`, dll.) ditempatkan di **Text Segment**.
- Dilakukan pemeriksaan sintaks, tipe data, dan alur logika.

3. Assembly

- Kode assembly diubah menjadi **object file (.o)**.
- Instruksi mesin dibuat untuk:
 - Loop (`for`, `while`)
 - Percabangan (`if`)
 - Pemanggilan fungsi pointer (`int (*alloc)(int,int)`)

4. Linking

- Object file digabung dengan library standar C (`glibc`).
- Simbol eksternal seperti `printf()` dan `clock()` dihubungkan.
- Dihasilkan file executable simulasi memori.

B. Proses Eksekusi

1. Inisialisasi Proses

- Program dijalankan dan OS membuat proses baru.
- Memory layout proses terbentuk:
 - **Text Segment** → kode program
 - **Data/BSS Segment** → array memory dan variabel global
 - **Stack** → variabel lokal fungsi
 - **Heap** → tidak digunakan (simulasi berbasis array)

2. Eksekusi Fungsi `main()`

- Program langsung menjalankan empat kali fungsi `benchmark()`:
 - First Fit
 - Best Fit
 - Worst Fit
 - Next Fit

3. Benchmark Dimulai (`benchmark`)

- Fungsi `init_memory(1000)` dipanggil:
 - Satu blok memori bebas berukuran 1000 dibuat.
 - `block_count = 1`
 - `last_alloc_index = 0`
- Waktu mulai dicatat menggunakan `clock()`.

4. Proses Alokasi Memori

- Fungsi alokasi dipanggil melalui **function pointer**:
 - `alloc(pid, size)`
- Mekanisme tiap algoritma:
 - **First Fit**: memilih blok bebas pertama yang cukup.
 - **Best Fit**: memilih blok bebas terkecil yang cukup.
 - **Worst Fit**: memilih blok bebas terbesar.
 - **Next Fit**: melanjutkan pencarian dari indeks terakhir.
- Jika ukuran blok lebih besar dari permintaan:
 - Blok dipecah (splitting).
 - `block_count` bertambah.

5. Dealokasi Memori

- Fungsi `deallocate(pid)`:
 - Menandai blok milik proses sebagai free.
 - Tidak melakukan penggabungan blok (coalescing).
- Hal ini menyebabkan **fragmentasi eksternal**.

6. Alokasi Ulang

- Proses baru dialokasikan setelah dealokasi.
- Algoritma kembali diuji pada kondisi memori terfragmentasi.

7. Pengukuran Waktu Eksekusi

- Waktu selesai dicatat (`clock()`).
- Execution time dihitung dalam mikrodetik (μs).
- Digunakan untuk membandingkan performa algoritma.

8. Perhitungan Fragmentasi

- Fungsi `calculate_fragmentation()` menghitung:
 - Total free space

- Largest free block
- Jumlah hole (blok bebas)
- External fragmentation (%)
- Menunjukkan efisiensi algoritma dalam mengelola memori.

9. Menampilkan Status Memori

- Fungsi display_memory() mencetak:
 - Setiap blok
 - Status (FREE atau Pid)
 - Ukuran blok
- Memvisualisasikan hasil akhir alokasi.

10. Pengulangan untuk Semua Algoritma

- Langkah 3–9 diulang untuk setiap algoritma.
- Hasil tiap algoritma dapat dibandingkan langsung.

11. Terminasi Program

- main() selesai dieksekusi.
- OS membersihkan seluruh resource proses.

3.3 TUGAS 3

3.3.1 Deskripsi Tugas

Tugas 3 berfokus pada pengembangan sistem paging lanjutan dengan menambahkan algoritma page replacement FIFO dan LRU pada program paging simulator. Mahasiswa harus menangani kondisi ketika seluruh frame memori telah penuh dan melakukan penggantian halaman secara otomatis. Selain implementasi algoritma, sistem juga dilengkapi dengan perhitungan statistik seperti jumlah page fault, hit ratio, dan disk writes untuk halaman yang dimodifikasi. Pengujian dilakukan menggunakan reference string tertentu dengan jumlah frame berbeda untuk membandingkan performa FIFO dan LRU. Analisis juga mencakup fenomena Belady's Anomaly serta evaluasi algoritma yang paling optimal dalam berbagai kondisi.

3.3.2 Source Code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>
#include <time.h>
```

```

#include <limits.h>

#define FRAME_SIZE 256
#define MAX_FRAMES 16
#define PAGE_SIZE FRAME_SIZE

// Algoritma yang tersedia
typedef enum {
    FIFO,
    LRU
} Algorithm;

typedef struct {
    int page_number;
    int frame_number;
    int valid;
    int reference_bit;
    int modify_bit;
    int last_used; // Untuk LRU - timestamp penggunaan terakhir
} PageTableEntry;

typedef struct {
    int occupied;
    int page_number;
    int load_time; // Untuk FIFO - waktu pemuatan halaman
    int dirty; // Untuk statistik - halaman telah dimodifikasi
} Frame;

// Variabel global
Frame physical_memory[MAX_FRAMES];
PageTableEntry page_table[100];
int num_pages = 0;
int num_frames = 4; // Default 4 frames untuk test scenario
Algorithm current_algorithm = FIFO;

// Statistik
int page_faults = 0;
int page_hits = 0;
int disk_writes = 0;
int total_accesses = 0;

// Untuk FIFO
int fifo_queue[MAX_FRAMES];
int front = 0, rear = -1, queue_count = 0;

// Untuk LRU
int lru_counter = 0;

void initialize_system(int frames) {
    num_frames = frames;

```

```

// Reset memory frames
for (int i = 0; i < MAX_FRAMES; i++) {
    physical_memory[i].occupied = 0;
    physical_memory[i].page_number = -1;
    physical_memory[i].load_time = 0;
    physical_memory[i].dirty = 0;
}

// Reset page table
for (int i = 0; i < 100; i++) {
    page_table[i].page_number = i;
    page_table[i].frame_number = -1;
    page_table[i].valid = 0;
    page_table[i].reference_bit = 0;
    page_table[i].modify_bit = 0;
    page_table[i].last_used = -1;
}

// Reset FIFO queue
front = 0;
rear = -1;
queue_count = 0;

// Reset LRU counter
lru_counter = 0;

// Reset statistik
page_faults = 0;
page_hits = 0;
disk_writes = 0;
total_accesses = 0;
}

void display_page_table() {
    printf("\n=== PAGE TABLE ===\n");
    printf("%-12s %-15s %-10s %-12s %-12s %-12s\n",
        "Page", "Frame", "Valid", "Referenced", "Modified", "Last Used");
    printf("-----\n");

    for (int i = 0; i < num_pages; i++) {
        printf("%-12d %-15d %-10s %-12s %-12s %-12d\n",
            page_table[i].page_number,
            page_table[i].frame_number,
            page_table[i].valid ? "Yes" : "No",
            page_table[i].reference_bit ? "Yes" : "No",
            page_table[i].modify_bit ? "Yes" : "No",
            page_table[i].last_used);
    }
    printf("\n");
}

```

```

void display_physical_memory() {
    printf("=== PHYSICAL MEMORY (%d FRAMES) ===\n", num_frames);
    printf("%-10s %-10s %-15s %-10s %-10s\n",
        "Frame", "Status", "Page Number", "Load Time", "Dirty");
    printf("-----\n");

    for (int i = 0; i < num_frames; i++) {
        printf("%-10d %-10s", i, physical_memory[i].occupied ? "Occupied" : "Free");
        if (physical_memory[i].occupied) {
            printf("%-15d %-10d %-10s",
                physical_memory[i].page_number,
                physical_memory[i].load_time,
                physical_memory[i].dirty ? "Yes" : "No");
        }
        printf("\n");
    }
    printf("\n");
}

int find_free_frame() {
    for (int i = 0; i < num_frames; i++) {
        if (!physical_memory[i].occupied) {
            return i;
        }
    }
    return -1;
}

void enqueue_fifo(int page_num) {
    rear = (rear + 1) % num_frames;
    fifo_queue[rear] = page_num;
    if (queue_count < num_frames) {
        queue_count++;
    }
}

int dequeue_fifo() {
    if (queue_count == 0) return -1;
    int page_num = fifo_queue[front];
    front = (front + 1) % num_frames;
    queue_count--;
    return page_num;
}

int find_lru_victim() {
    int victim_frame = -1;
    int min_last_used = INT_MAX;

    for (int i = 0; i < num_frames; i++) {
        if (physical_memory[i].occupied) {
            int page_num = physical_memory[i].page_number;

```

```

        if (page_table[page_num].last_used < min_last_used) {
            min_last_used = page_table[page_num].last_used;
            victim_frame = i;
        }
    }
}
return victim_frame;
}

int find_fifo_victim() {
    // FIFO menggunakan queue, jadi kita ambil dari depan
    if (queue_count == 0) return -1;

    int victim_page = fifo_queue[front];
    for (int i = 0; i < num_frames; i++) {
        if (physical_memory[i].occupied && physical_memory[i].page_number == victim_page) {
            return i;
        }
    }
    return -1;
}

int replace_page(int page_num, int write) {
    int victim_frame = -1;

    if (current_algorithm == FIFO) {
        victim_frame = find_fifo_victim();
    } else if (current_algorithm == LRU) {
        victim_frame = find_lru_victim();
    }

    if (victim_frame == -1) {
        return -1;
    }

    int victim_page = physical_memory[victim_frame].page_number;

    // Jika halaman korban telah dimodifikasi, perlu write ke disk
    if (page_table[victim_page].modify_bit && physical_memory[victim_frame].dirty) {
        disk_writes++;
        printf(" [DISK WRITE] Page %d (dirty) ditulis ke disk\n", victim_page);
    }

    // Update page table untuk halaman korban
    page_table[victim_page].valid = 0;
    page_table[victim_page].frame_number = -1;
    page_table[victim_page].modify_bit = 0;

    // Update page table untuk halaman baru
    page_table[page_num].frame_number = victim_frame;
    page_table[page_num].valid = 1;

```

```

page_table[page_num].reference_bit = 1;
page_table[page_num].last_used = lru_counter++;
if (write) {
    page_table[page_num].modify_bit = 1;
}

// Update physical memory
physical_memory[victim_frame].page_number = page_num;
physical_memory[victim_frame].load_time = total_accesses;
physical_memory[victim_frame].dirty = write ? 1 : 0;

// Update FIFO queue jika menggunakan FIFO
if (current_algorithm == FIFO) {
    dequeue_fifo();
    enqueue_fifo(page_num);
}

return victim_frame;
}

int allocate_page(int page_num, int write) {
    total_accesses++;

    if (page_num >= 100) {
        printf("Error: Page number terlalu besar\n");
        return -1;
    }

    // Jika halaman sudah ada di memori (HIT)
    if (page_table[page_num].valid) {
        page_hits++;
        page_table[page_num].reference_bit = 1;
        page_table[page_num].last_used = lru_counter++;

        if (write) {
            page_table[page_num].modify_bit = 1;
            // Update dirty bit di physical memory
            int frame_num = page_table[page_num].frame_number;
            if (frame_num >= 0 && frame_num < num_frames) {
                physical_memory[frame_num].dirty = 1;
            }
        }

        return page_table[page_num].frame_number;
    }

    // PAGE FAULT
    page_faults++;

    int frame = find_free_frame();

```



```

if (frame == -1) {
    // Tidak ada frame kosong, perlu page replacement
    printf(" [PAGE REPLACEMENT] Menggunakan algoritma %s\n",
        current_algorithm == FIFO ? "FIFO" : "LRU");
    frame = replace_page(page_num, write);
    if (frame == -1) {
        printf("Error: Gagal melakukan page replacement\n");
        return -1;
    }
    printf(" Page %d menggantikan page di frame %d\n", page_num, frame);
} else {
    // Ada frame kosong
    page_table[page_num].frame_number = frame;
    page_table[page_num].valid = 1;
    page_table[page_num].reference_bit = 1;
    page_table[page_num].last_used = lru_counter++;
    if (write) {
        page_table[page_num].modify_bit = 1;
    }

    physical_memory[frame].occupied = 1;
    physical_memory[frame].page_number = page_num;
    physical_memory[frame].load_time = total_accesses;
    physical_memory[frame].dirty = write ? 1 : 0;

    // Untuk FIFO, tambahkan ke queue
    if (current_algorithm == FIFO) {
        enqueue_fifo(page_num);
    }

    printf("Page %d dialokasikan ke frame %d\n", page_num, frame);
}

if (page_num >= num_pages) {
    num_pages = page_num + 1;
}

return frame;
}

void display_statistics() {
    printf("\n=== STATISTIK ===\n");
    printf("Algoritma: %s\n", current_algorithm == FIFO ? "FIFO" : "LRU");
    printf("Jumlah Frames: %d\n", num_frames);
    printf("Total Akses: %d\n", total_accesses);
    printf("Page Faults: %d\n", page_faults);
    printf("Page Hits: %d\n", page_hits);

    if (total_accesses > 0) {
        float fault_rate = (float)page_faults / total_accesses * 100;
        float hit_rate = (float)page_hits / total_accesses * 100;
    }
}

```

```

    printf("Page Fault Rate: %.2f%%\n", fault_rate);
    printf("Hit Rate: %.2f%%\n", hit_rate);
}

printf("Disk Writes: %d\n", disk_writes);
printf("\n");
}

void run_test_scenario(int frames, Algorithm algo) {

printf("\n=====
===== \n");
    printf("|| TEST SCENARIO - %s dengan %d frames || \n",
        algo == FIFO ? "FIFO" : "LRU", frames);

printf("||
===== \n\n");

    initialize_system(frames);
    current_algorithm = algo;

    // Reference string dari soal
    int reference_string[] = {7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1};
    int length = sizeof(reference_string) / sizeof(reference_string[0]);

    // Untuk menampilkan trace table
    printf("Step\tReference\tFrame 0\tFrame 1\tFrame 2\tFrame 3\tFault?\n");
    printf("-----\n");

    for (int i = 0; i < length; i++) {
        int page_num = reference_string[i];
        int fault_before = page_faults;

        // Simulasi akses read
        allocate_page(page_num, 0);

        // Tampilkan trace
        printf("%d\t%d\t\t", i+1, page_num);

        // Tampilkan isi frames
        for (int f = 0; f < frames; f++) {
            if (physical_memory[f].occupied) {
                printf("%d\t", physical_memory[f].page_number);
            } else {
                printf("-\t");
            }
        }

        // Tampilkan apakah terjadi fault
        if (page_faults > fault_before) {
            printf("FAULT");

```

```

    } else {
        printf("HIT");
    }
    printf("\n");
}

display_statistics();
}

```

```

void compare_algorithms() {

```

```

    printf("\n
=====
===== \n");
    printf(" || PERBANDINGAN ALGORITMA (4 frames) || \n");

    printf("
=====
===== \n\n");

```

```

    printf("%-15s %-20s %-15s %-20s\n",
           "Algoritma", "Jumlah Page Fault", "Hit Ratio", "Disk Writes");
    printf("-----\n");

```

```

    // Jalankan FIFO
    run_test_scenario(4, FIFO);
    int fifo_faults = page_faults;
    int fifo_writes = disk_writes;
    float fifo_hit_rate = (float)page_hits / total_accesses * 100;

```

```

    // Jalankan LRU
    run_test_scenario(4, LRU);
    int lru_faults = page_faults;
    int lru_writes = disk_writes;
    float lru_hit_rate = (float)page_hits / total_accesses * 100;

```

```

    printf("\n=== TABEL PERBANDINGAN ===\n");
    printf("%-15s %-20d %-15.2f%% %-20d\n",
           "FIFO", fifo_faults, fifo_hit_rate, fifo_writes);
    printf("%-15s %-20d %-15.2f%% %-20d\n",
           "LRU", lru_faults, lru_hit_rate, lru_writes);
}

```

```

void belady_anomaly_test() {

```

```

    printf("\n
=====
===== \n");
    printf(" || BELADY'S ANOMALY TEST || \n");

    printf("
=====
===== \n\n");

```

```

    printf("Menguji apakah page fault berkurang seiring bertambahnya frames:\n\n");

```

```
// Test untuk 3, 4, dan 5 frames
int frames_list[] = {3, 4, 5};
int num_tests = sizeof(frames_list) / sizeof(frames_list[0]);

printf("%-10s %-15s %-15s\n", "Frames", "FIFO Faults", "LRU Faults");
printf("-----\n");

for (int i = 0; i < num_tests; i++) {
    int frames = frames_list[i];

    // Test FIFO
    run_test_scenario(frames, FIFO);
    int fifo_faults = page_faults;

    // Test LRU
    run_test_scenario(frames, LRU);
    int lru_faults = page_faults;

    printf("%-10d %-15d %-15d\n", frames, fifo_faults, lru_faults);
}

printf("\nANALISIS:\n");
printf("Belady's Anomaly terjadi ketika jumlah page fault BERTAMBAH\n");
printf("meskipun jumlah frame ditambah. Fenomena ini bisa terjadi\n");
printf("pada algoritma FIFO tetapi tidak pada LRU.\n");
}

void demo_scenario() {

printf("\n=====
===== \n");
printf(" || DEMO SCENARIO: PAGING SYSTEM || \n");

printf(" ||=====
===== \n\n");

initialize_system(4);
current_algorithm = FIFO;

printf("Skenario: Program berukuran 2048 bytes\n");
printf("Jumlah pages: %d pages\n\n", 2048 / PAGE_SIZE);

// Simulasi beberapa akses
int accesses[][2] = {
    {0, 0}, // Read address 0
    {300, 1}, // Write address 300
    {512, 0}, // Read address 512
    {100, 0}, // Read address 100
    {1024, 1} // Write address 1024
};
};
```

```

for (int i = 0; i < 5; i++) {
    int addr = accesses[i][0];
    int write = accesses[i][1];

    printf("\nAkses %s pada logical address %d:\n",
        write ? "WRITE" : "READ", addr);

    int page_num = addr / PAGE_SIZE;
    int frame = allocate_page(page_num, write);

    if (frame != -1) {
        printf(" Page %d -> Frame %d\n", page_num, frame);
    }
}

display_page_table();
display_physical_memory();
display_statistics();
}

int main() {
    int choice, address, write, frames, algo_choice;

    // Default initialization
    initialize_system(4);

    while(1) {
        printf("\n=== ADVANCED PAGING SIMULATOR ===\n");
        printf("1. Set Jumlah Frames\n");
        printf("2. Set Algoritma (FIFO/LRU)\n");
        printf("3. Allocate Page\n");
        printf("4. Access Memory (Read)\n");
        printf("5. Access Memory (Write)\n");
        printf("6. Display Page Table\n");
        printf("7. Display Physical Memory\n");
        printf("8. Display Statistics\n");
        printf("9. Run Demo Scenario\n");
        printf("10. Run Test Scenario (Reference String)\n");
        printf("11. Compare FIFO vs LRU\n");
        printf("12. Belady's Anomaly Test\n");
        printf("13. Reset System\n");
        printf("14. Exit\n");
        printf("Pilihan: ");
        scanf("%d", &choice);

        switch(choice) {
            case 1:
                printf("Masukkan jumlah frames (1-%d): ", MAX_FRAMES);
                scanf("%d", &frames);
                if (frames > 0 && frames <= MAX_FRAMES) {

```

```

        initialize_system(frames);
        printf("Sistem direset dengan %d frames\n", frames);
    } else {
        printf("Jumlah frames tidak valid!\n");
    }
    break;

case 2:
    printf("Pilih algoritma:\n");
    printf("1. FIFO\n");
    printf("2. LRU\n");
    printf("Pilihan: ");
    scanf("%d", &algo_choice);
    if (algo_choice == 1) {
        current_algorithm = FIFO;
        printf("Algoritma diubah menjadi FIFO\n");
    } else if (algo_choice == 2) {
        current_algorithm = LRU;
        printf("Algoritma diubah menjadi LRU\n");
    } else {
        printf("Pilihan tidak valid!\n");
    }
    break;

case 3:
    printf("Masukkan page number: ");
    scanf("%d", &address);
    printf("Tipe akses (0=Read, 1=Write): ");
    scanf("%d", &write);
    allocate_page(address, write);
    break;

case 4:
    printf("Masukkan logical address: ");
    scanf("%d", &address);
    printf("Page %d diakses (READ)\n", address / PAGE_SIZE);
    allocate_page(address / PAGE_SIZE, 0);
    break;

case 5:
    printf("Masukkan logical address: ");
    scanf("%d", &address);
    printf("Page %d diakses (WRITE)\n", address / PAGE_SIZE);
    allocate_page(address / PAGE_SIZE, 1);
    break;

case 6:
    display_page_table();
    break;

case 7:

```

```

        display_physical_memory();
        break;

    case 8:
        display_statistics();
        break;

    case 9:
        demo_scenario();
        break;

    case 10:
        printf("Masukkan jumlah frames untuk test: ");
        scanf("%d", &frames);
        printf("Pilih algoritma (1=FIFO, 2=LRU): ");
        scanf("%d", &algo_choice);
        if (frames > 0 && frames <= MAX_FRAMES && (algo_choice == 1 || algo_choice == 2)) {
            run_test_scenario(frames, algo_choice == 1 ? FIFO : LRU);
        } else {
            printf("Input tidak valid!\n");
        }
        break;

    case 11:
        compare_algorithms();
        break;

    case 12:
        belady_anomaly_test();
        break;
    Screenshot Output
    case 13:
        initialize_system(num_frames);
        printf("Sistem direset\n");
        break;

    case 14:
        printf("Keluar dari program...\n");
        return 0;

    default:
        printf("Pilihan tidak valid!\n");
    }
}

return 0;
}

```

3.3.3 Screenshot Output

```
rio_rege@DESKTOP-NVPM5HQ:~/praktikum-memory/paging$ ./paging_simulator
```

```
=== ADVANCED PAGING SIMULATOR ===
```

1. Set Jumlah Frames
2. Set Algoritma (FIFO/LRU)
3. Allocate Page
4. Access Memory (Read)
5. Access Memory (Write)
6. Display Page Table
7. Display Physical Memory
8. Display Statistics
9. Run Demo Scenario
10. Run Test Scenario (Reference String)
11. Compare FIFO vs LRU
12. Belady's Anomaly Test
13. Reset System
14. Exit

Pilihan: 10

Masukkan jumlah frames untuk test: 4

Pilih algoritma (1=FIFO, 2=LRU): 1

TEST SCENARIO - FIFO dengan 4 frames

Step	Reference	Frame 0	Frame 1	Frame 2	Frame 3	Fault?
Page 7 dialokasikan ke frame 0						
1	7	-	-	-	-	FAULT
Page 0 dialokasikan ke frame 1						
2	0	7	0	-	-	FAULT
Page 1 dialokasikan ke frame 2						
3	1	7	0	1	-	FAULT
Page 2 dialokasikan ke frame 3						
4	2	7	0	1	2	FAULT
5	0	7	0	1	2	HIT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO						
Page 3 menggantikan page di frame 0						
6	3	3	0	1	2	FAULT
7	0	3	0	1	2	HIT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO						
Page 4 menggantikan page di frame 1						
8	4	3	4	1	2	FAULT
9	2	3	4	1	2	HIT
10	3	3	4	1	2	HIT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO						
Page 0 menggantikan page di frame 2						
11	0	3	4	0	2	FAULT
12	3	3	4	0	2	HIT
13	2	3	4	0	2	HIT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO						
Page 1 menggantikan page di frame 3						
14	1	3	4	0	1	FAULT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO						
Page 2 menggantikan page di frame 0						
15	2	4	0	0	1	FAULT
16	0	2	4	0	1	HIT
17	1	2	4	0	1	HIT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO						
Page 7 menggantikan page di frame 1						
18	7	2	7	0	1	FAULT
19	0	2	7	0	1	HIT
20	1	2	7	0	1	HIT

```
=== STATISTIK ===
```

Algoritma: FIFO
Jumlah Frames: 4
Total Akses: 20
Page Faults: 10
Page Hits: 10
Page Fault Rate: 50.00%
Hit Rate: 50.00%
Disk Writes: 0

```
=== ADVANCED PAGING SIMULATOR ===
```

1. Set Jumlah Frames
2. Set Algoritma (FIFO/LRU)
3. Allocate Page
4. Access Memory (Read)
5. Access Memory (Write)
6. Display Page Table
7. Display Physical Memory
8. Display Statistics
9. Run Demo Scenario
10. Run Test Scenario (Reference String)
11. Compare FIFO vs LRU
12. Belady's Anomaly Test
13. Reset System
14. Exit

Pilihan: 10

Masukkan jumlah frames untuk test: 4

Pilih algoritma (1=FIFO, 2=LRU): 2

TEST SCENARIO - LRU dengan 4 frames

Step	Reference	Frame 0	Frame 1	Frame 2	Frame 3	Fault?
Page 7 dialokasikan ke frame 0						
1	7	-	-	-	-	FAULT
Page 0 dialokasikan ke frame 1						
2	0	7	0	-	-	FAULT
Page 1 dialokasikan ke frame 2						
3	1	7	0	1	-	FAULT
Page 2 dialokasikan ke frame 3						
4	2	7	0	1	2	FAULT
5	0	7	0	1	2	HIT
[PAGE REPLACEMENT] Menggunakan algoritma LRU						
Page 3 menggantikan page di frame 0						
6	3	3	0	1	2	FAULT
7	0	3	0	1	2	HIT
[PAGE REPLACEMENT] Menggunakan algoritma LRU						
Page 4 menggantikan page di frame 2						
8	4	3	0	4	2	FAULT
9	2	3	0	4	2	HIT
10	3	3	0	4	2	HIT
11	0	3	0	4	2	HIT
12	3	3	0	4	2	HIT
13	2	3	0	4	2	HIT
[PAGE REPLACEMENT] Menggunakan algoritma LRU						
Page 1 menggantikan page di frame 2						
14	1	3	0	1	2	FAULT
15	2	3	0	1	2	HIT
16	0	3	0	1	2	HIT
17	1	3	0	1	2	HIT
[PAGE REPLACEMENT] Menggunakan algoritma LRU						
Page 7 menggantikan page di frame 0						
18	7	7	0	1	2	FAULT
19	0	7	0	1	2	HIT
20	1	7	0	1	2	HIT


```
=== STATISTIK ===
Algoritma: LRU
Jumlah Frames: 4
Total Akses: 20
Page Faults: 8
Page Hits: 12
Page Fault Rate: 40.00%
Hit Rate: 60.00%
Disk Writes: 0
```

=== ADVANCED PAGING SIMULATOR ===

1. Set Jumlah Frames
 2. Set Algoritma (FIFO/LRU)
 3. Allocate Page
 4. Access Memory (Read)
 5. Access Memory (Write)
 6. Display Page Table
 7. Display Physical Memory
 8. Display Statistics
 9. Run Demo Scenario
 10. Run Test Scenario (Reference String)
 11. Compare FIFO vs LRU
 12. Belady's Anomaly Test
 13. Reset System
 14. Exit
- Pilihan: 11

PERBANDINGAN ALGORITMA (4 frames)

Algoritma	Jumlah Page Fault	Hit Ratio	Disk Writes
-----------	-------------------	-----------	-------------

TEST SCENARIO - FIFO dengan 4 frames

Step	Reference	Frame 0	Frame 1	Frame 2	Frame 3	Fault?
Page 7 dialokasikan ke frame 0						
1	7	-	-	-	-	FAULT
Page 0 dialokasikan ke frame 1						
2	0	7	0	-	-	FAULT
Page 1 dialokasikan ke frame 2						
3	1	7	0	1	-	FAULT
Page 2 dialokasikan ke frame 3						
4	2	7	0	1	2	FAULT
5	0	7	0	1	2	HIT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO						
Page 3 menggantikan page di frame 0						
6	3	3	0	1	2	FAULT
7	0	3	0	1	2	HIT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO						
Page 4 menggantikan page di frame 1						
8	4	3	4	1	2	FAULT
9	2	3	4	1	2	HIT
10	3	3	4	1	2	HIT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO						
Page 0 menggantikan page di frame 2						
11	0	3	4	0	2	FAULT
12	3	3	4	0	2	HIT
13	2	3	4	0	2	HIT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO						
Page 1 menggantikan page di frame 3						

14	1	3	4	0	1	FAULT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO						
Page 2 menggantikan page di frame 0						
15	2	2	4	0	1	FAULT
16	0	2	4	0	1	HIT
17	1	2	4	0	1	HIT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO						
Page 7 menggantikan page di frame 1						
18	7	2	7	0	1	FAULT
19	0	2	7	0	1	HIT
20	1	2	7	0	1	HIT

```
=== STATISTIK ===
Algoritma: FIFO
Jumlah Frames: 4
Total Akses: 20
Page Faults: 10
Page Hits: 10
Page Fault Rate: 50.00%
Hit Rate: 50.00%
Disk Writes: 0
```

TEST SCENARIO - LRU dengan 4 frames

Step	Reference	Frame 0	Frame 1	Frame 2	Frame 3	Fault?
Page 7 dialokasikan ke frame 0						
1	7	-	-	-	-	FAULT
Page 0 dialokasikan ke frame 1						
2	0	7	0	-	-	FAULT
Page 1 dialokasikan ke frame 2						
3	1	7	0	1	-	FAULT
Page 2 dialokasikan ke frame 3						
4	2	7	0	1	2	FAULT
5	0	7	0	1	2	HIT
[PAGE REPLACEMENT] Menggunakan algoritma LRU						
Page 3 menggantikan page di frame 0						
6	3	3	0	1	2	FAULT
7	0	3	0	1	2	HIT
[PAGE REPLACEMENT] Menggunakan algoritma LRU						
Page 4 menggantikan page di frame 2						
8	4	3	0	4	2	FAULT
9	2	3	0	4	2	HIT
10	3	3	0	4	2	HIT
11	0	3	0	4	2	HIT
12	3	3	0	4	2	HIT
13	3	3	0	4	2	HIT
[PAGE REPLACEMENT] Menggunakan algoritma LRU						
Page 1 menggantikan page di frame 2						
14	1	3	0	1	2	FAULT
15	2	3	0	1	2	HIT
16	0	3	0	1	2	HIT
17	1	3	0	1	2	HIT
[PAGE REPLACEMENT] Menggunakan algoritma LRU						
Page 7 menggantikan page di frame 0						
18	7	7	0	1	2	FAULT
19	0	7	0	1	2	HIT
20	1	7	0	1	2	HIT

```
=== STATISTIK ===
Algoritma: LRU
Jumlah Frames: 4
```

Total Akses: 20
Page Faults: 8
Page Hits: 12
Page Fault Rate: 40.00%
Hit Rate: 60.00%
Disk Writes: 0

=== TABEL PERBANDINGAN ===

FIFO	10	50.00	% 0
LRU	8	60.00	% 0

=== ADVANCED PAGING SIMULATOR ===

1. Set Jumlah Frames
 2. Set Algoritma (FIFO/LRU)
 3. Allocate Page
 4. Access Memory (Read)
 5. Access Memory (Write)
 6. Display Page Table
 7. Display Physical Memory
 8. Display Statistics
 9. Run Demo Scenario
 10. Run Test Scenario (Reference String)
 11. Compare FIFO vs LRU
 12. Belady's Anomaly Test
 13. Reset System
 14. Exit
- Pilihan: 12

BELADY'S ANOMALY TEST

Menguji apakah page fault berkurang seiring bertambahnya frames:

Frames	FIFO Faults	LRU Faults
--------	-------------	------------

TEST SCENARIO - FIFO dengan 3 frames

Step	Reference	Frame 0	Frame 1	Frame 2	Frame 3	Fault?
------	-----------	---------	---------	---------	---------	--------

Page 7 dialokasikan ke frame 0
1 7 7 - - FAULT
Page 0 dialokasikan ke frame 1
2 0 7 0 - FAULT
Page 1 dialokasikan ke frame 2
3 1 7 0 1 FAULT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO
Page 2 menggantikan page di frame 0
4 2 2 0 1 FAULT
5 0 2 0 1 HIT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO
Page 3 menggantikan page di frame 1
6 3 2 3 1 FAULT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO
Page 0 menggantikan page di frame 2
7 0 2 3 0 FAULT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO
Page 4 menggantikan page di frame 0
8 4 4 3 0 FAULT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO
Page 2 menggantikan page di frame 1
9 2 4 2 0 FAULT

10 3 4 2 3 FAULT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO
Page 0 menggantikan page di frame 0
11 0 0 2 3 FAULT
12 3 0 2 3 HIT
13 2 0 2 3 HIT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO
Page 1 menggantikan page di frame 1
14 1 0 1 3 FAULT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO
Page 2 menggantikan page di frame 2
15 2 0 1 2 FAULT
16 0 0 1 2 HIT
17 1 0 1 2 HIT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO
Page 7 menggantikan page di frame 0
18 7 7 1 2 FAULT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO
Page 0 menggantikan page di frame 1
19 0 7 0 2 FAULT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO
Page 1 menggantikan page di frame 2
20 1 7 0 1 FAULT

=== STATISTIK ===
Algoritma: FIFO
Jumlah Frames: 3
Total Akses: 20
Page Faults: 15
Page Hits: 5
Page Fault Rate: 75.00%
Hit Rate: 25.00%
Disk Writes: 0

TEST SCENARIO - LRU dengan 3 frames

Step	Reference	Frame 0	Frame 1	Frame 2	Frame 3	Fault?
------	-----------	---------	---------	---------	---------	--------

Page 7 dialokasikan ke frame 0
1 7 7 - - FAULT
Page 0 dialokasikan ke frame 1
2 0 7 0 - FAULT
Page 1 dialokasikan ke frame 2
3 1 7 0 1 FAULT
[PAGE REPLACEMENT] Menggunakan algoritma LRU
Page 2 menggantikan page di frame 0
4 2 2 0 1 FAULT
5 0 2 0 1 HIT
[PAGE REPLACEMENT] Menggunakan algoritma LRU
Page 3 menggantikan page di frame 2
6 3 2 0 3 FAULT
7 0 2 0 3 HIT
[PAGE REPLACEMENT] Menggunakan algoritma LRU
Page 4 menggantikan page di frame 0
8 4 4 0 3 FAULT
[PAGE REPLACEMENT] Menggunakan algoritma LRU
Page 2 menggantikan page di frame 2
9 2 4 0 2 FAULT
[PAGE REPLACEMENT] Menggunakan algoritma LRU
Page 3 menggantikan page di frame 1
10 3 4 3 2 FAULT
[PAGE REPLACEMENT] Menggunakan algoritma LRU
Page 0 menggantikan page di frame 0

```
11 0 0 3 2 FAULT
12 3 0 3 2 HIT
13 2 0 3 2 HIT
[PAGE REPLACEMENT] Menggunakan algoritma LRU
Page 1 menggantikan page di frame 0
14 1 1 3 2 FAULT
15 7 1 3 2 HIT
[PAGE REPLACEMENT] Menggunakan algoritma LRU
Page 0 menggantikan page di frame 1
16 0 1 0 2 FAULT
17 1 1 0 2 HIT
[PAGE REPLACEMENT] Menggunakan algoritma LRU
Page 7 menggantikan page di frame 2
18 7 1 0 7 FAULT
19 0 1 0 7 HIT
20 1 1 0 7 HIT
```

```
=== STATISTIK ===
Algoritma: LRU
Jumlah Frames: 3
Total Akses: 20
Page Faults: 12
Page Hits: 8
Page Fault Rate: 60.00%
Hit Rate: 40.00%
Disk Writes: 0
```

```
3 15 12
```

TEST SCENARIO - FIFO dengan 4 frames

Step	Reference	Frame 0	Frame 1	Frame 2	Frame 3	Fault?
Page 7 dialokasikan ke frame 0						
1	7	-	-	-	-	FAULT
Page 0 dialokasikan ke frame 1						
2	0	7	0	-	-	FAULT
Page 1 dialokasikan ke frame 2						
3	1	7	0	1	-	FAULT
Page 2 dialokasikan ke frame 3						
4	2	7	0	1	2	FAULT
5	0	7	0	1	2	HIT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO						
Page 3 menggantikan page di frame 0						
6	3	3	0	1	2	FAULT
7	0	3	0	1	2	HIT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO						
Page 4 menggantikan page di frame 1						
8	4	3	4	1	2	FAULT
9	2	3	4	1	2	HIT
10	3	3	4	1	2	HIT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO						
Page 0 menggantikan page di frame 2						
11	0	3	4	0	2	FAULT
12	3	3	4	0	2	HIT
13	2	3	4	0	2	HIT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO						
Page 1 menggantikan page di frame 3						
14	1	3	4	0	1	FAULT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO						
Page 2 menggantikan page di frame 0						
15	2	2	4	0	1	FAULT
16	0	2	4	0	1	HIT
17	1	2	4	0	1	HIT

```

[PAGE REPLACEMENT] Menggunakan algoritma FIFO
Page 7 menggantikan page di frame 1
18 7 2 7 0 1 FAULT
19 0 2 7 0 1 HIT
20 1 2 7 0 1 HIT

```

```

=== STATISTIK ===
Algoritma: FIFO
Jumlah Frames: 4
Total Akses: 20
Page Faults: 10
Page Hits: 10
Page Fault Rate: 50.00%
Hit Rate: 50.00%
Disk Writes: 0

```

TEST SCENARIO - LRU dengan 4 frames

Step	Reference	Frame 0	Frame 1	Frame 2	Frame 3	Fault?
Page 7 dialokasikan ke frame 0						
1	7	7	-	-	-	FAULT
Page 0 dialokasikan ke frame 1						
2	0	7	0	-	-	FAULT
Page 1 dialokasikan ke frame 2						
3	1	7	0	1	-	FAULT
Page 2 dialokasikan ke frame 3						
4	2	7	0	1	2	FAULT
5	0	7	0	1	2	HIT
[PAGE REPLACEMENT] Menggunakan algoritma LRU						
Page 3 menggantikan page di frame 0						
6	3	3	0	1	2	FAULT
7	0	3	0	1	2	HIT
[PAGE REPLACEMENT] Menggunakan algoritma LRU						
Page 4 menggantikan page di frame 2						
8	4	3	0	4	2	FAULT
9	2	3	0	4	2	HIT
10	3	3	0	4	2	HIT
11	0	3	0	4	2	HIT
12	3	3	0	4	2	HIT
13	2	3	0	4	2	HIT
[PAGE REPLACEMENT] Menggunakan algoritma LRU						
Page 1 menggantikan page di frame 2						
14	1	3	0	1	2	FAULT
15	2	3	0	1	2	HIT
16	0	3	0	1	2	HIT
17	1	3	0	1	2	HIT
[PAGE REPLACEMENT] Menggunakan algoritma LRU						
Page 7 menggantikan page di frame 0						
18	7	7	0	1	2	FAULT
19	0	7	0	1	2	HIT
20	1	7	0	1	2	HIT

```

=== STATISTIK ===
Algoritma: LRU
Jumlah Frames: 4
Total Akses: 20
Page Faults: 8
Page Hits: 12
Page Fault Rate: 40.00%
Hit Rate: 60.00%
Disk Writes: 0

```

TEST SCENARIO - FIFO dengan 5 frames

Step	Reference	Frame 0	Frame 1	Frame 2	Frame 3	Fault?	
Page 7 dialokasikan ke frame 0							
1	7	7	-	-	-	FAULT	
Page 0 dialokasikan ke frame 1							
2	0	7	0	-	-	FAULT	
Page 1 dialokasikan ke frame 2							
3	1	7	0	1	-	FAULT	
Page 2 dialokasikan ke frame 3							
4	2	7	0	1	2	FAULT	
5	0	7	0	1	2	HIT	
Page 3 dialokasikan ke frame 4							
6	3	7	0	1	2	3	FAULT
7	0	7	0	1	2	3	HIT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO							
Page 4 menggantikan page di frame 0							
8	4	4	0	1	2	3	FAULT
9	2	4	0	1	2	3	HIT
10	3	4	0	1	2	3	HIT
11	0	4	0	1	2	3	HIT
12	3	4	0	1	2	3	HIT
13	2	4	0	1	2	3	HIT
14	1	4	0	1	2	3	HIT
15	2	4	0	1	2	3	HIT
16	0	4	0	1	2	3	HIT
17	1	4	0	1	2	3	HIT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO							
Page 7 menggantikan page di frame 1							
18	7	4	7	1	2	3	FAULT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO							
Page 0 menggantikan page di frame 2							
19	0	4	7	0	2	3	FAULT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO							
Page 1 menggantikan page di frame 3							
20	1	4	7	0	1	3	FAULT

=== STATISTIK ===
 Algoritma: FIFO
 Jumlah Frames: 5
 Total Akses: 20
 Page Faults: 9
 Page Hits: 11
 Page Fault Rate: 45.00%
 Hit Rate: 55.00%
 Disk Writes: 0

TEST SCENARIO - LRU dengan 5 frames

Step	Reference	Frame 0	Frame 1	Frame 2	Frame 3	Fault?
Page 7 dialokasikan ke frame 0						
1	7	7	-	-	-	FAULT
Page 0 dialokasikan ke frame 1						
2	0	7	0	-	-	FAULT
Page 1 dialokasikan ke frame 2						
3	1	7	0	1	-	FAULT

```

2      0      7      0      -      -      -      FAULT
Page 1 dialokasikan ke frame 2
3      1      7      0      1      -      -      FAULT
Page 2 dialokasikan ke frame 3
4      2      7      0      1      2      -      FAULT
5      0      7      0      1      2      -      HIT
Page 3 dialokasikan ke frame 4
6      3      7      0      1      2      3      FAULT
7      0      7      0      1      2      3      HIT
[PAGE REPLACEMENT] Menggunakan algoritma LRU
Page 4 menggantikan page di frame 0
8      4      4      0      1      2      3      FAULT
9      2      4      0      1      2      3      HIT
10     3      4      0      1      2      3      HIT
11     0      4      0      1      2      3      HIT
12     3      4      0      1      2      3      HIT
13     2      4      0      1      2      3      HIT
14     1      4      0      1      2      3      HIT
15     2      4      0      1      2      3      HIT
16     0      4      0      1      2      3      HIT
17     1      4      0      1      2      3      HIT
[PAGE REPLACEMENT] Menggunakan algoritma LRU
Page 7 menggantikan page di frame 0
18     7      7      0      1      2      3      FAULT
19     0      7      0      1      2      3      HIT
20     1      7      0      1      2      3      HIT

=== STATISTIK ===
Algoritma: LRU
Jumlah Frames: 5
Total Akses: 20
Page Faults: 7
Page Hits: 13
Page Fault Rate: 35.00%
Hit Rate: 65.00%
Disk Writes: 0

5      9      7

ANALISIS:
Belady's Anomaly terjadi ketika jumlah page fault BERTAMBAH
meskipun jumlah frame ditambah. Fenomena ini bisa terjadi
pada algoritma FIFO tetapi tidak pada LRU.

=== ADVANCED PAGING SIMULATOR ===
1. Set Jumlah Frames
2. Set Algoritma (FIFO/LRU)
3. Allocate Page
4. Access Memory (Read)
5. Access Memory (Write)
6. Display Page Table
7. Display Physical Memory
8. Display Statistics
9. Run Demo Scenario
10. Run Test Scenario (Reference String)
11. Compare FIFO vs LRU
12. Belady's Anomaly Test
13. Reset System
14. Exit
Pilihan: 14
Keluar dari program...
rio_regex@DESKTOP-NVPM5HQ:~/praktikum-memory/paging$ |

```

3.3.4 Penjelasan Implementasi

A. Proses Kompilasi Program

1. Preprocessing

- Compiler membaca semua **header file**:
 - `stdio.h` → fungsi input/output (`printf`, `scanf`)
 - `stdlib.h` → alokasi memori & utilitas umum
 - `string.h` → manipulasi string
 - `math.h` → fungsi matematika (disiapkan, meski tidak dominan dipakai)
 - `time.h` → penanganan waktu
 - `limits.h` → konstanta batas tipe data (`INT_MAX`)
- **Makro** diproses:
 - `FRAME_SIZE`, `MAX_FRAMES`, `PAGE_SIZE`

- Semua #define diganti dengan nilainya secara literal.
- Komentar dihapus dan file menjadi satu source siap dikompilasi.

2. Compilation (Penerjemahan ke Assembly)

- Kode C diterjemahkan ke **bahasa assembly**.
- Struktur data diterjemahkan:
 - struct Frame
 - struct PageTableEntry
- Fungsi-fungsi seperti:
 - initialize_system
 - allocate_page
 - replace_page
 - run_test_scenario
diterjemahkan menjadi instruksi mesin tingkat rendah.
- Variabel global seperti:
 - physical_memory
 - page_table
 - statistik (page_faults, page_hits)
dialokasikan di **data segment / BSS segment**.

3. Assembly

- Kode assembly diubah menjadi **object file (.o)**.
- Referensi fungsi eksternal (misalnya printf, scanf) masih belum terhubung.

4. Linking

- Object file digabung dengan **library standar C**:
 - libc untuk I/O, memory, dll.
- Semua simbol eksternal diselesaikan.
- Menghasilkan **file executable** (misalnya paging_simulator).

B. Proses Eksekusi Program

1. Program Dimulai (main)

- Sistem operasi:
 - Membuat **process baru**
 - Mengalokasikan **virtual address space**

- Memuat program ke memori
- Fungsi main() dipanggil pertama kali.

2. Inisialisasi Sistem

- initialize_system(4) dipanggil:
 - Semua frame fisik dikosongkan
 - Page table di-reset
 - FIFO queue & counter LRU di-reset
 - Statistik diatur ke nol
- Menyiapkan **lingkungan simulasi paging**.

3. Menu Interaktif

- Program masuk ke **loop tak hingga (while(1))**
- Pengguna dapat:
 - Mengatur jumlah frame
 - Memilih algoritma FIFO atau LRU
 - Melakukan akses memory (READ / WRITE)
 - Menjalankan skenario uji & demo

4. Akses Memori (allocate_page)

Saat pengguna atau skenario mengakses halaman:

- total_accesses bertambah
- Program mengecek **page table**:
 - **HIT** → halaman sudah ada di memori
 - Update reference bit
 - Update last_used (untuk LRU)
 - **PAGE FAULT** → halaman tidak ada
 - Cari frame kosong
 - Jika penuh → jalankan **page replacement**
 - FIFO → halaman paling awal masuk diganti
 - LRU → halaman paling lama tidak dipakai diganti
- Jika halaman dirty:
 - Simulasi **penulisan ke disk**
 - disk_writes bertambah

5. Page Replacement

- `replace_page()`:
 - Menentukan victim frame sesuai algoritma
 - Update page table halaman korban
 - Update frame dengan halaman baru
 - FIFO queue atau LRU timestamp diperbarui

6. Simulasi & Visualisasi

- Fungsi seperti:
 - `display_page_table()`
 - `display_physical_memory()`
 - `display_statistics()`
menampilkan kondisi internal sistem paging.
- `run_test_scenario()`:
 - Menjalankan reference string
 - Menampilkan **trace table** step-by-step
- `belady_anomaly_test()`:
 - Menguji fenomena **Belady's Anomaly**
 - Membandingkan FIFO dan LRU

7. Terminasi Program

- Saat memilih **Exit**:
 - Program keluar dari loop
 - `return 0` dieksekusi
- Sistem operasi:
 - Membebaskan memori proses
 - Mengakhiri eksekusi program

3.4 TUGAS 4

3.4.1 Deskripsi Tugas

Tugas 4 bertujuan untuk memahami dan membandingkan berbagai struktur page table dalam sistem operasi. Mahasiswa diminta mengimplementasikan three-level page table dan inverted page table, kemudian menganalisis perbedaan keduanya dari segi overhead memori dan kecepatan translasi alamat. Dalam laporan, mahasiswa menghitung jumlah entri, ukuran tabel, total penggunaan memori,

serta efisiensi ruang dibandingkan single-level page table. Selain itu, dianalisis jumlah akses memori yang dibutuhkan untuk translasi alamat dan peran TLB dalam meningkatkan performa. Studi kasus penggunaan struktur page table pada sistem 32-bit dan 64-bit juga dibahas secara konseptual.

3.4.2 Source Code

```
#include <stdio.h>
#include <stdlib.h>

#define L1_BITS 16
#define L2_BITS 8
#define L3_BITS 8
#define OFFSET_BITS 12

#define NUM_FRAMES 1024 // untuk inverted page table

/* ===== */
/* THREE-LEVEL PAGE TABLE */
/* ===== */

typedef struct {
    int frame_number;
    int valid;
    int read, write, execute;
} L3_Entry;

typedef struct {
    L3_Entry *entries;
    int valid;
} L2_Entry;

typedef struct {
    L2_Entry *entries;
    int valid;
} L1_Entry;

L1_Entry *L1;

/* Ambil index dari virtual address */
unsigned int get_l1(unsigned long va) {
    return (va >> (L2_BITS + L3_BITS + OFFSET_BITS)) & ((1 << L1_BITS) - 1);
}

unsigned int get_l2(unsigned long va) {
    return (va >> (L3_BITS + OFFSET_BITS)) & ((1 << L2_BITS) - 1);
}

unsigned int get_l3(unsigned long va) {
    return (va >> OFFSET_BITS) & ((1 << L3_BITS) - 1);
}
```

```

/* Alokasi page */
void allocate_3level(unsigned long va, int frame) {
    int l1 = get_l1(va);
    int l2 = get_l2(va);
    int l3 = get_l3(va);

    if (!L1[l1].valid) {
        L1[l1].entries = calloc(1 << L2_BITS, sizeof(L2_Entry));
        L1[l1].valid = 1;
    }

    if (!L1[l1].entries[l2].valid) {
        L1[l1].entries[l2].entries = calloc(1 << L3_BITS, sizeof(L3_Entry));
        L1[l1].entries[l2].valid = 1;
    }

    L1[l1].entries[l2].entries[l3].frame_number = frame;
    L1[l1].entries[l2].entries[l3].valid = 1;
}

/* Translasi alamat */
int translate_3level(unsigned long va) {
    int l1 = get_l1(va);
    int l2 = get_l2(va);
    int l3 = get_l3(va);
    int offset = va & ((1 << OFFSET_BITS) - 1);

    if (!L1[l1].valid) return -1;
    if (!L1[l1].entries[l2].valid) return -1;
    if (!L1[l1].entries[l2].entries[l3].valid) return -1;

    return (L1[l1].entries[l2].entries[l3].frame_number << OFFSET_BITS) | offset;
}

/* Hitung memory overhead */
void calculate_memory_overhead() {
    int l1_count = 1 << L1_BITS;
    int l2_count = 0, l3_count = 0;

    for (int i = 0; i < l1_count; i++) {
        if (L1[i].valid) {
            l2_count++;
            for (int j = 0; j < (1 << L2_BITS); j++) {
                if (L1[i].entries[j].valid) {
                    l3_count++;
                }
            }
        }
    }
}

printf("\n=== MEMORY OVERHEAD THREE-LEVEL ===\n");

```

```

printf("L1 tables : %d\n", l1_count);
printf("L2 tables aktif : %d\n", l2_count);
printf("L3 tables aktif : %d\n", l3_count);
}

/* ===== */
/* INVERTED PAGE TABLE      */
/* ===== */

typedef struct {
    int process_id;
    int page_number;
    int valid;
} InvertedPageEntry;

InvertedPageEntry inverted_table[NUM_FRAMES];

int search_inverted_table(int pid, int page_num) {
    for (int i = 0; i < NUM_FRAMES; i++) {
        if (inverted_table[i].valid &&
            inverted_table[i].process_id == pid &&
            inverted_table[i].page_number == page_num) {
            return i;
        }
    }
    return -1;
}

void insert_inverted_table(int pid, int page_num, int frame) {
    inverted_table[frame].process_id = pid;
    inverted_table[frame].page_number = page_num;
    inverted_table[frame].valid = 1;
}

/* ===== */
/* MAIN PROGRAM      */
/* ===== */

int main() {
    unsigned long va1 = 0x123456789ABC;
    unsigned long va2 = 0xABCDEF123456;

    /* Inisialisasi L1 */
    L1 = calloc(1 << L1_BITS, sizeof(L1_Entry));

    /* Alokasi page */
    allocate_3level(va1, 10);
    allocate_3level(va2, 25);

    /* Translasi alamat */
    int pa1 = translate_3level(va1);

```

```

int pa2 = translate_3level(va2);

printf("Virtual Address 1 : %lx -> Physical Address : %x\n", va1, pa1);
printf("Virtual Address 2 : %lx -> Physical Address : %x\n", va2, pa2);

calculate_memory_overhead();

/* Inverted Page Table */
insert_inverted_table(1, 5, 100);
insert_inverted_table(2, 8, 200);

printf("\n=== INVERTED PAGE TABLE ===\n");
printf("PID 1 Page 5 -> Frame %d\n",
        search_inverted_table(1, 5));
printf("PID 2 Page 8 -> Frame %d\n",
        search_inverted_table(2, 8));

return 0;
}

```

3.4.3 Screenshot Output

```

rio_regex@DESKTOP-NVPM5HQ:~/praktikum-memory/page-table$ nano two_level_paging.c
rio_regex@DESKTOP-NVPM5HQ:~/praktikum-memory/page-table$ gcc -o two_level_paging two_level_paging.c
rio_regex@DESKTOP-NVPM5HQ:~/praktikum-memory/page-table$ ./two_level_paging
Virtual Address 1 : 123456789abc -> Physical Address : aabc
Virtual Address 2 : abcdef123456 -> Physical Address : 19456

=== MEMORY OVERHEAD THREE-LEVEL ===
L1 tables : 65536
L2 tables aktif : 2
L3 tables aktif : 2

=== INVERTED PAGE TABLE ===
PID 1 Page 5 -> Frame 100
PID 2 Page 8 -> Frame 200
rio_regex@DESKTOP-NVPM5HQ:~/praktikum-memory/page-table$ |

```

3.4.4 Penjelasan Implementasi

A. Proses Kompilasi

1. Preprocessing

- Compiler membaca header:
 - `stdio.h` → fungsi input/output (`printf`)
 - `stdlib.h` → alokasi memori dinamis (`calloc`)
- Direktif `#define` diproses:
 - `L1_BITS`, `L2_BITS`, `L3_BITS`, `OFFSET_BITS`
 - `NUM_FRAMES`
- Semua makro diganti dengan nilai konstanta.
- Komentar dihapus dan kode disiapkan menjadi satu unit source.

2. Compilation (C → Assembly)

- Kode C diterjemahkan ke **assembly language**.
- Struktur data dialokasikan:
 - L1_Entry, L2_Entry, L3_Entry
 - InvertedPageEntry
- Variabel global:
 - Pointer L1
 - Array inverted_table
ditempatkan di **data segment / BSS segment**.
- Fungsi-fungsi seperti:
 - allocate_3level
 - translate_3level
 - search_inverted_table
dikonversi menjadi instruksi mesin.

3. Assembly

- Kode assembly diubah menjadi **object file (.o)**.
- Referensi fungsi eksternal (printf, calloc) masih bersifat simbolik.

4. Linking

- Object file digabung dengan **C standard library (libc)**.
- Semua simbol eksternal diselesaikan.
- Dihasilkan **file executable** yang siap dijalankan.

B. Proses Eksekusi

1. Program Dimulai

- Sistem operasi:
 - Membuat **process baru**
 - Mengalokasikan ruang alamat virtual
- Fungsi main() mulai dieksekusi.

2. Inisialisasi Three-Level Page Table

- $L1 = \text{calloc}(1 \ll L1_BITS, \text{sizeof}(L1_Entry));$
- Dialokasikan **Level-1 Page Table**:
 - Semua entri awal bernilai valid = 0
- Alokasi dilakukan di **heap**.

3. Alokasi Halaman (Three-Level Page Table)

- `allocate_3level(va1, 10)` dan `allocate_3level(va2, 25)`:
 - Virtual address dipecah menjadi:
 - **L1 index**
 - **L2 index**
 - **L3 index**
 - **offset**
 - Jika tabel L1 atau L2 belum ada:
 - Dialokasikan secara **on-demand** menggunakan `calloc`
 - Entri L3 diisi:
 - `frame_number`
 - `valid = 1`

4. Translasi Alamat Virtual ke Fisik

- `translate_3level(va)`:
 - Mengecek validitas:
 - $L1 \rightarrow L2 \rightarrow L3$
 - Jika salah satu tidak valid $\rightarrow$ return -1
 - Jika valid:
 - $\text{Physical Address} = (\text{frame_number} \ll \text{OFFSET_BITS}) \mid \text{offset}$
- Hasil translasi dicetak ke layar.

5. Perhitungan Memory Overhead

- `calculate_memory_overhead()`:
 - Menghitung jumlah tabel:
 - Total L1
 - L2 aktif
 - L3 aktif
- Menunjukkan **efisiensi ruang** pada skema multi-level page table.

6. Inverted Page Table

- `insert_inverted_table(pid, page, frame)`:
 - Menyimpan mapping **(PID, page) $\rightarrow$ frame**
- `search_inverted_table(pid, page)`:
 - Melakukan pencarian linear

- Mengembalikan nomor frame jika ditemukan
- Hasil ditampilkan ke layar.

7. Terminasi Program

- return 0; dieksekusi.
- Sistem operasi:
 - Membersihkan heap
 - Mengakhiri process

3.5 TUGAS 5

3.5.1 Deskripsi Tugas

Tugas 5 mensimulasikan mekanisme swapping pada sistem operasi dengan berbagai strategi pemilihan proses korban. Mahasiswa diminta mengimplementasikan simulasi swap in dan swap out menggunakan strategi FIFO, LRU, dan priority-based. Simulasi dijalankan dengan skenario memori fisik terbatas dan sejumlah proses dengan ukuran bervariasi yang melebihi kapasitas memori. Selama simulasi, dilakukan pencatatan metrik performa seperti jumlah swap in, swap out, total waktu swapping, dan rata-rata response time. Hasil pengujian dianalisis untuk menentukan strategi yang paling efektif dalam meminimalkan thrashing serta mengevaluasi kapan swapping lebih baik dibandingkan menghentikan proses.

3.5.2 Source Code

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <unistd.h>

#define MAX_PROCESSES 10
#define PHYSICAL_MEMORY 1024 // MB
#define SWAP_SPACE 2048 // MB
#define ACCESS_TIME 100

typedef enum { IN_MEMORY, SWAPPED } State;

typedef struct {
    int pid;
    int size;
    int priority;
    State state;
    time_t load_time;
    time_t last_access;
```



```

    int swap_in_count;
    int swap_out_count;
} Process;

Process processes[MAX_PROCESSES];
int used_memory = 0;
int used_swap = 0;

/* ===== METRICS ===== */
int total_swap_in = 0;
int total_swap_out = 0;
long total_swap_time = 0;

/* ===== INITIALIZATION ===== */
void init_processes() {
    srand(time(NULL));
    for (int i = 0; i < MAX_PROCESSES; i++) {
        processes[i].pid = i + 1;
        processes[i].size = 50 + rand() % 151; // 50–200 MB
        processes[i].priority = 1 + rand() % 10;
        processes[i].state = SWAPPED;
        processes[i].swap_in_count = 0;
        processes[i].swap_out_count = 0;
    }
}

/* ===== VICTIM SELECTION ===== */
int select_fifo() {
    time_t oldest = time(NULL);
    int victim = -1;
    for (int i = 0; i < MAX_PROCESSES; i++) {
        if (processes[i].state == IN_MEMORY &&
            processes[i].load_time < oldest) {
            oldest = processes[i].load_time;
            victim = i;
        }
    }
    return victim;
}

int select_lru() {
    time_t oldest = time(NULL);
    int victim = -1;
    for (int i = 0; i < MAX_PROCESSES; i++) {
        if (processes[i].state == IN_MEMORY &&
            processes[i].last_access < oldest) {
            oldest = processes[i].last_access;
            victim = i;
        }
    }
    return victim;
}

```

```

}

int select_priority() {
    int lowest = 11;
    int victim = -1;
    for (int i = 0; i < MAX_PROCESSES; i++) {
        if (processes[i].state == IN_MEMORY &&
            processes[i].priority < lowest) {
            lowest = processes[i].priority;
            victim = i;
        }
    }
    return victim;
}

/* ===== SWAP OPERATIONS ===== */
void swap_out(int idx) {
    if (idx < 0) return;

    usleep(processes[idx].size * 1000);
    total_swap_time += processes[idx].size;

    used_memory -= processes[idx].size;
    used_swap += processes[idx].size;

    processes[idx].state = SWAPPED;
    processes[idx].swap_out_count++;
    total_swap_out++;
}

void swap_in(int idx) {
    usleep(processes[idx].size * 1000);
    total_swap_time += processes[idx].size;

    used_memory += processes[idx].size;
    used_swap -= processes[idx].size;

    processes[idx].state = IN_MEMORY;
    processes[idx].load_time = time(NULL);
    processes[idx].last_access = time(NULL);
    processes[idx].swap_in_count++;
    total_swap_in++;
}

/* ===== LOAD PROCESS ===== */
void load_process(int idx, int strategy) {
    if (processes[idx].state == IN_MEMORY) return;

    while (used_memory + processes[idx].size > PHYSICAL_MEMORY) {
        int victim = -1;
        if (strategy == 1) victim = select_fifo();
    }
}

```

```

        else if (strategy == 2) victim = select_lru();
        else victim = select_priority();

        if (victim == -1) break;
        swap_out(victim);
    }

    swap_in(idx);
}

/* ===== ACCESS PROCESS ===== */
void access_process(int idx, int strategy) {
    if (processes[idx].state == SWAPPED) {
        load_process(idx, strategy);
    }
    processes[idx].last_access = time(NULL);
}

/* ===== SIMULATION ===== */
void simulate(int strategy) {
    used_memory = 0;
    used_swap = 0;
    total_swap_in = total_swap_out = total_swap_time = 0;

    for (int i = 0; i < MAX_PROCESSES; i++)
        processes[i].state = SWAPPED;

    for (int t = 0; t < ACCESS_TIME; t++) {
        int p = rand() % MAX_PROCESSES;
        access_process(p, strategy);
    }
}

/* ===== REPORT ===== */
void report(char *name) {
    printf("\n=== %s STRATEGY ===\n", name);
    printf("Swap In Count : %d\n", total_swap_in);
    printf("Swap Out Count : %d\n", total_swap_out);
    printf("Total Swap Time: %ld ms\n", total_swap_time);
    printf("Avg Response   : %.2f ms\n",
        (double)total_swap_time / ACCESS_TIME);
}

/* ===== MAIN ===== */
int main() {
    init_processes();

    simulate(1);
    report("FIFO");

    simulate(2);

```

```

report("LRU");

simulate(3);
report("PRIORITY");

return 0;
}

```

3.5.3 Screenshot Output

```

rio_regex@DESKTOP-NVPMHQ:~/praktikum-memory/swapping$ nano swap_simulator.c
rio_regex@DESKTOP-NVPMHQ:~/praktikum-memory/swapping$ gcc -o swap_simulator swap_simulator.c
rio_regex@DESKTOP-NVPMHQ:~/praktikum-memory/swapping$ ./swap_simulator

=== FIFO STRATEGY ===
Swap In Count : 17
Swap Out Count : 8
Total Swap Time: 2682 ms
Avg Response : 26.82 ms

=== LRU STRATEGY ===
Swap In Count : 10
Swap Out Count : 0
Total Swap Time: 1056 ms
Avg Response : 10.56 ms

=== PRIORITY STRATEGY ===
Swap In Count : 15
Swap Out Count : 6
Total Swap Time: 2399 ms
Avg Response : 23.99 ms
rio_regex@DESKTOP-NVPMHQ:~/praktikum-memory/swapping$ |

```

3.5.4 Penjelasan Implementasi

A. Proses Kompilasi

1. Preprocessing

- Compiler membaca header:
 - `stdio.h` → fungsi output (`printf`)
 - `stdlib.h` → alokasi & fungsi acak (`rand`, `srand`)
 - `time.h` → manajemen waktu (`time`, `time_t`)
 - `unistd.h` → fungsi delay (`usleep`)
- Direktif `#define` diproses:
 - `MAX_PROCESSES`, `PHYSICAL_MEMORY`, `SWAP_SPACE`, `ACCESS_TIME`
- Enum State dan struct Process didefinisikan.
- Komentar dihapus dan kode siap dikompilasi.

2. Compilation (C → Assembly)

- Kode C diterjemahkan menjadi **assembly language**.
- Struktur Process dialokasikan:
 - `processes[MAX_PROCESSES]` disimpan di **global memory**
- Variabel global:

- used_memory, used_swap
- total_swap_in, total_swap_out, total_swap_time
- Fungsi-fungsi seperti:
 - swap_in, swap_out
 - select_fifo, select_lru, select_priority diterjemahkan ke instruksi mesin.

3. Assembly

- Kode assembly diubah menjadi **object file (.o)**.
- Referensi ke fungsi library (printf, usleep, time) masih berupa simbol.

4. Linking

- Object file digabung dengan **standard C library**.
- Semua simbol eksternal diselesaikan.
- Dihasilkan **file executable** yang siap dijalankan.

B. Proses Eksekusi

1. Program Dimulai

- Sistem operasi membuat **process baru**.
- Fungsi main() mulai dieksekusi.

2. Inisialisasi Proses

- Fungsi init_processes() dipanggil:
 - Setiap proses diberi:
 - pid
 - ukuran memori acak (50–200 MB)
 - prioritas acak (1–10)
 - Semua proses awalnya berada di **SWAPPED**
- Seed random diatur dengan srand(time(NULL)).

3. Simulasi dengan Strategi FIFO

- simulate(1) dijalankan:
 - Reset penggunaan memori dan swap
 - Semua proses diset ke SWAPPED
- Selama ACCESS_TIME (100 kali):
 - Proses dipilih secara acak

- Jika belum di memori → dilakukan **swap-in**
- Jika memori penuh → dipilih korban dengan **FIFO**
- Setiap swap:
 - Disimulasikan delay dengan usleep
 - Waktu swap dicatat

4. Pelaporan FIFO

- Fungsi report("FIFO"):
 - Menampilkan:
 - Total swap-in
 - Total swap-out
 - Total waktu swap
 - Rata-rata waktu respon

5. Simulasi dengan Strategi LRU

- simulate(2) dijalankan:
 - Mekanisme sama seperti FIFO
 - Korban dipilih berdasarkan **last_access paling lama**
- Menggambarkan prinsip **Least Recently Used**.

6. Pelaporan LRU

- Statistik LRU ditampilkan menggunakan report("LRU").

7. Simulasi dengan Strategi PRIORITY

- simulate(3) dijalankan:
 - Korban dipilih berdasarkan **prioritas terendah**
- Mensimulasikan kebijakan berbasis **prioritas proses**.

8. Pelaporan PRIORITY

- Statistik swapping untuk PRIORITY ditampilkan.

9. Program Berakhir

- return 0; dieksekusi.
- Sistem operasi:
 - Menghentikan process
 - Membersihkan resource

3.6 TUGAS 6

3.6.1 Deskripsi Tugas

Tugas 6 bertujuan untuk menganalisis arsitektur memori pada sistem nyata dengan menggunakan perintah sistem pada Linux. Mahasiswa diminta mengumpulkan informasi terkait arsitektur prosesor, ukuran page size, TLB, serta hierarki cache (L1, L2, dan L3). Data yang diperoleh kemudian dibandingkan dengan karakteristik arsitektur Intel 32-bit, Intel 64-bit, dan ARMv8. Dalam analisis, mahasiswa menjelaskan jumlah bit address space yang digunakan, implementasi multi-level paging pada arsitektur tersebut, serta kapasitas maksimum virtual memory. Tugas ini menghubungkan konsep teori manajemen memori dengan implementasi nyata pada perangkat keras modern.

3.6.2 Source Code

```
# 1. Cek informasi CPU
lscpu | grep -E "Architecture|CPU|Model|Thread|Core"

# 2. Cek page size
getconf PAGE_SIZE

# 3. Cek TLB info
cpuid | grep -i tlb

# 4. Cek cache hierarchy
lscpu | grep -i cache

# 1. Total memory
free -h

# 2. Virtual memory info
vmstat 1 5

# 3. Memory map dari proses
cat /proc/self/maps

# 4. Swap usage
swapon --show
```

3.6.3 Screenshot Output

```
rio_regex@DESKTOP-NVPM5HQ:~/praktikum-memory$ lscpu | grep -E "Architecture|CPU|Model|Thread|Core"
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
CPU(s):                12
On-line CPU(s) list:   0-11
Model name:             AMD Ryzen 5 6600H with Radeon Graphics
CPU family:            25
Model:                 68
Thread(s) per core:    2
Core(s) per socket:    6
NUMA node0 CPU(s):    0-11
```

```
rio_regex@DESKTOP-NVPMHQ:~/praktikum-memory$ getconf PAGE_SIZE
4096
```

```
rio_regex@DESKTOP-NVPMHQ:~/praktikum-memory$ cpuid | grep -i tlb
cache and TLB information (2):
  use hypercalls for local TLB flushes      = false
  use hypercalls for remote TLB flushes     = true
  nested enlightened TLB flush support      = true
L1 TLB/cache information: 2M/4M pages & L1 TLB (0x80000005/eax):
L1 TLB/cache information: 4K pages & L1 TLB (0x80000005/ebx):
L2 TLB/cache information: 2M/4M pages & L2 TLB (0x80000006/eax):
L2 TLB/cache information: 4K pages & L2 TLB (0x80000006/ebx):
  INVLPGB supports TLB flush guest nested = false
  INVLPGB/TLBSYNC hyperv interc enable    = false
L1 TLB information: 1G pages (0x80000019/eax):
L2 TLB information: 1G pages (0x80000019/ebx):
cache and TLB information (2):
  use hypercalls for local TLB flushes      = false
  use hypercalls for remote TLB flushes     = true
  nested enlightened TLB flush support      = true
L1 TLB/cache information: 2M/4M pages & L1 TLB (0x80000005/eax):
L1 TLB/cache information: 4K pages & L1 TLB (0x80000005/ebx):
L2 TLB/cache information: 2M/4M pages & L2 TLB (0x80000006/eax):
L2 TLB/cache information: 4K pages & L2 TLB (0x80000006/ebx):
  INVLPGB supports TLB flush guest nested = false
  INVLPGB/TLBSYNC hyperv interc enable    = false
L1 TLB information: 1G pages (0x80000019/eax):
L2 TLB information: 1G pages (0x80000019/ebx):
cache and TLB information (2):
  use hypercalls for local TLB flushes      = false
  use hypercalls for remote TLB flushes     = true
  nested enlightened TLB flush support      = true
L1 TLB/cache information: 2M/4M pages & L1 TLB (0x80000005/eax):
L1 TLB/cache information: 4K pages & L1 TLB (0x80000005/ebx):
L2 TLB/cache information: 2M/4M pages & L2 TLB (0x80000006/eax):
L2 TLB/cache information: 4K pages & L2 TLB (0x80000006/ebx):
  INVLPGB supports TLB flush guest nested = false
  INVLPGB/TLBSYNC hyperv interc enable    = false
L1 TLB information: 1G pages (0x80000019/eax):
L2 TLB information: 1G pages (0x80000019/ebx):
cache and TLB information (2):
  use hypercalls for local TLB flushes      = false
  use hypercalls for remote TLB flushes     = true
  nested enlightened TLB flush support      = true
L1 TLB/cache information: 2M/4M pages & L1 TLB (0x80000005/eax):
L1 TLB/cache information: 4K pages & L1 TLB (0x80000005/ebx):
L2 TLB/cache information: 2M/4M pages & L2 TLB (0x80000006/eax):
L2 TLB/cache information: 4K pages & L2 TLB (0x80000006/ebx):
  INVLPGB supports TLB flush guest nested = false
  INVLPGB/TLBSYNC hyperv interc enable    = false
L1 TLB information: 1G pages (0x80000019/eax):
L2 TLB information: 1G pages (0x80000019/ebx):
cache and TLB information (2):
```



```

use hypercalls for local TLB flushes      = false
use hypercalls for remote TLB flushes     = true
nested enlightened TLB flush support      = true
L1 TLB/cache information: 2M/4M pages & L1 TLB (0x80000005/eax):
L1 TLB/cache information: 4K pages & L1 TLB (0x80000005/ebx):
L2 TLB/cache information: 2M/4M pages & L2 TLB (0x80000006/eax):
L2 TLB/cache information: 4K pages & L2 TLB (0x80000006/ebx):
INVLPGB supports TLB flush guest nested = false
INVLPGB/TLBSYNC hyperv interc enable    = false
L1 TLB information: 1G pages (0x80000019/eax):
L2 TLB information: 1G pages (0x80000019/ebx):
cache and TLB information (2):
use hypercalls for local TLB flushes      = false
use hypercalls for remote TLB flushes     = true
nested enlightened TLB flush support      = true
L1 TLB/cache information: 2M/4M pages & L1 TLB (0x80000005/eax):
L1 TLB/cache information: 4K pages & L1 TLB (0x80000005/ebx):
L2 TLB/cache information: 2M/4M pages & L2 TLB (0x80000006/eax):
L2 TLB/cache information: 4K pages & L2 TLB (0x80000006/ebx):
INVLPGB supports TLB flush guest nested = false
INVLPGB/TLBSYNC hyperv interc enable    = false
L1 TLB information: 1G pages (0x80000019/eax):
L2 TLB information: 1G pages (0x80000019/ebx):
cache and TLB information (2):
use hypercalls for local TLB flushes      = false
use hypercalls for remote TLB flushes     = true
nested enlightened TLB flush support      = true
L1 TLB/cache information: 2M/4M pages & L1 TLB (0x80000005/eax):
L1 TLB/cache information: 4K pages & L1 TLB (0x80000005/ebx):
L2 TLB/cache information: 2M/4M pages & L2 TLB (0x80000006/eax):
L2 TLB/cache information: 4K pages & L2 TLB (0x80000006/ebx):
INVLPGB supports TLB flush guest nested = false
INVLPGB/TLBSYNC hyperv interc enable    = false
L1 TLB information: 1G pages (0x80000019/eax):
L2 TLB information: 1G pages (0x80000019/ebx):
cache and TLB information (2):
use hypercalls for local TLB flushes      = false
use hypercalls for remote TLB flushes     = true
nested enlightened TLB flush support      = true
L1 TLB/cache information: 2M/4M pages & L1 TLB (0x80000005/eax):
L1 TLB/cache information: 4K pages & L1 TLB (0x80000005/ebx):
L2 TLB/cache information: 2M/4M pages & L2 TLB (0x80000006/eax):
L2 TLB/cache information: 4K pages & L2 TLB (0x80000006/ebx):

```

```

INVLPGB supports TLB flush guest nested = false
INVLPGB/TLBSYNC hyperv interc enable    = false
L1 TLB information: 1G pages (0x80000019/eax):
L2 TLB information: 1G pages (0x80000019/ebx):
cache and TLB information (2):
use hypercalls for local TLB flushes      = false
use hypercalls for remote TLB flushes     = true
nested enlightened TLB flush support      = true
L1 TLB/cache information: 2M/4M pages & L1 TLB (0x80000005/eax):
L1 TLB/cache information: 4K pages & L1 TLB (0x80000005/ebx):
L2 TLB/cache information: 2M/4M pages & L2 TLB (0x80000006/eax):
L2 TLB/cache information: 4K pages & L2 TLB (0x80000006/ebx):
INVLPGB supports TLB flush guest nested = false
INVLPGB/TLBSYNC hyperv interc enable    = false
L1 TLB information: 1G pages (0x80000019/eax):
L2 TLB information: 1G pages (0x80000019/ebx):
rio_regex@DESKTOP-NVPM5HQ:~/praktikum-memory$ |

```

```

rio_regex@DESKTOP-NVPM5HQ:~/praktikum-memory$ /usr/bin/lscpu | /bin/grep -i cache
L1d cache:                192 KiB (6 instances)
L1i cache:                192 KiB (6 instances)
L2 cache:                 3 MiB (6 instances)
L3 cache:                 16 MiB (1 instance)
rio_regex@DESKTOP-NVPM5HQ:~/praktikum-memory$ |

```

```

rio_regex@DESKTOP-NVPM5HQ:~/praktikum-memory$ free -h
              total        used        free      shared  buff/cache   available
Mem:          7.4Gi        538Mi        6.8Gi        3.6Mi        190Mi        6.9Gi
Swap:         2.0Gi          0B        2.0Gi
rio_regex@DESKTOP-NVPM5HQ:~/praktikum-memory$ |

```

```

rio_regex@DESKTOP-NVPM5HQ:~/praktikum-memory$ vmstat 1 5
procs -----memory-----swap-----io-----system-----cpu-----
r  b   swpd   free   buff   cache   si   so    bi   bo    in   cs   us   sy   id   wa   st   gu
0  0       0 7154212   5468 189956   0    0    68   41   88    0    0    0 100    0    0    0
0  0       0 7154212   5468 189956   0    0    0   116   70   42    0    0 100    0    0    0
0  0       0 7154212   5468 190128   0    0    0    60   50    0    0 100    0    0    0
0  0       0 7154212   5468 190128   0    0    0    219  72    0    0 100    0    0    0
0  0       0 7154212   5468 190128   0    0    0    81   64    0    0 100    0    0    0
rio_regex@DESKTOP-NVPM5HQ:~/praktikum-memory$ |

rio_regex@DESKTOP-NVPM5HQ:~/praktikum-memory$ cat /proc/self/maps
5854dc424000-5854dc426000 r--p 00000000 08:30 14788                /usr/bin/cat
5854dc426000-5854dc42b000 r-xp 00002000 08:30 14788                /usr/bin/cat
5854dc42b000-5854dc42d000 r--p 00007000 08:30 14788                /usr/bin/cat
5854dc42d000-5854dc42e000 r--p 00008000 08:30 14788                /usr/bin/cat
5854dc42e000-5854dc42f000 rw-p 00009000 08:30 14788                /usr/bin/cat
58550f66b000-58550f68c000 rw-p 00000000 00:00 0                    [heap]
7cfa87dde000-7cfa87e00000 rw-p 00000000 00:00 0
7cfa87e00000-7cfa87e28000 r--p 00000000 08:30 46704                /usr/lib/x86_64-linux-gnu/libc.so.6
7cfa87e28000-7cfa87fb0000 r-xp 00028000 08:30 46704                /usr/lib/x86_64-linux-gnu/libc.so.6
7cfa87fb0000-7cfa87fff000 r--p 001b0000 08:30 46704                /usr/lib/x86_64-linux-gnu/libc.so.6
7cfa87fff000-7cfa88003000 r--p 001fe000 08:30 46704                /usr/lib/x86_64-linux-gnu/libc.so.6
7cfa88003000-7cfa88005000 rw-p 00202000 08:30 46704                /usr/lib/x86_64-linux-gnu/libc.so.6
7cfa88005000-7cfa88012000 rw-p 00000000 00:00 0
7cfa88012000-7cfa88089000 r--p 00000000 08:30 12230                /usr/lib/locale/C.utf8/LC_CTYPE
7cfa88089000-7cfa8808a000 r--p 00000000 08:30 12236                /usr/lib/locale/C.utf8/LC_NUMERIC
7cfa8808a000-7cfa8808b000 r--p 00000000 08:30 12239                /usr/lib/locale/C.utf8/LC_TIME
7cfa8808b000-7cfa8808c000 r--p 00000000 08:30 12229                /usr/lib/locale/C.utf8/LC_COLLATE
7cfa8808c000-7cfa8808d000 r--p 00000000 08:30 12234                /usr/lib/locale/C.utf8/LC_MONETARY
7cfa8808d000-7cfa8808e000 r--p 00000000 08:30 12233                /usr/lib/locale/C.utf8/LC_MESSAGES/SYS_LC_MESSAGES
7cfa8808e000-7cfa8808f000 r--p 00000000 08:30 12237                /usr/lib/locale/C.utf8/LC_PAPER
7cfa8808f000-7cfa88096000 r--s 00000000 08:30 46693                /usr/lib/x86_64-linux-gnu/gconv/gconv-modules.cache
7cfa88096000-7cfa88099000 rw-p 00000000 00:00 0
7cfa88099000-7cfa8809a000 r--p 00000000 08:30 12235                /usr/lib/locale/C.utf8/LC_NAME
7cfa8809a000-7cfa8809b000 r--p 00000000 08:30 12228                /usr/lib/locale/C.utf8/LC_ADDRESS
7cfa8809b000-7cfa8809c000 r--p 00000000 08:30 12238                /usr/lib/locale/C.utf8/LC_TELEPHONE
7cfa8809c000-7cfa8809d000 r--p 00000000 08:30 12232                /usr/lib/locale/C.utf8/LC_MEASUREMENT
7cfa8809d000-7cfa8809e000 r--p 00000000 08:30 12231                /usr/lib/locale/C.utf8/LC_IDENTIFICATION
7cfa8809e000-7cfa880a0000 rw-p 00000000 00:00 0
7cfa880a0000-7cfa880a1000 r--p 00000000 08:30 46701                /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7cfa880a1000-7cfa880cc000 r-xp 00001000 08:30 46701                /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7cfa880cc000-7cfa880d6000 r--p 0002c000 08:30 46701                /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7cfa880d6000-7cfa880d8000 r--p 00036000 08:30 46701                /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7cfa880d8000-7cfa880da000 rw-p 00038000 08:30 46701                /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7ffc9d32c000-7ffc9d34d000 rw-p 00000000 00:00 0                    [stack]
7ffc9d34d000-7ffc9d3fb000 r--p 00000000 00:00 0                    [vvar]
7ffc9d3fb000-7ffc9d3fd000 r-xp 00000000 00:00 0                    [vdso]
rio_regex@DESKTOP-NVPM5HQ:~/praktikum-memory$ |

rio_regex@DESKTOP-NVPM5HQ:~/praktikum-memory$ swapon --show
NAME      TYPE      SIZE USED PRIO
/dev/sdc  partition 2G      0B    -2
rio_regex@DESKTOP-NVPM5HQ:~/praktikum-memory$ |

```

3.6.4 Penjelasan Implementasi

- Skrip dijalankan oleh shell (Bash) dan dieksekusi langsung tanpa proses kompilasi.
- Setiap perintah dijalankan secara berurutan dari baris pertama hingga terakhir.
- Pengecekan Informasi CPU
- Perintah lscpu digunakan untuk menampilkan informasi detail tentang prosesor.
- Output difilter dengan grep untuk menampilkan data penting seperti arsitektur CPU, model prosesor, jumlah core, dan thread.
- Informasi ini membantu memahami kemampuan arsitektur sistem (32-bit atau 64-bit).
- Pengecekan Ukuran Halaman Memori

- Perintah `getconf PAGE_SIZE` digunakan untuk mengetahui ukuran halaman memori standar sistem.
- Umumnya bernilai 4096 byte (4 KB).
- Ukuran halaman menentukan pembagian offset pada alamat virtual dalam mekanisme paging.
- Pengecekan Informasi TLB
- Perintah `cpuid` digunakan untuk menampilkan fitur perangkat keras CPU.
- Output difilter untuk menampilkan informasi Translation Lookaside Buffer (TLB).
- TLB berperan mempercepat proses translasi alamat virtual ke alamat fisik.
- Pengecekan Hirarki Cache
- Perintah `lscpu | grep -i cache` menampilkan informasi cache CPU (L1, L2, dan L3).
- Hirarki cache memengaruhi kecepatan akses memori dan performa sistem secara keseluruhan.
- Skrip ini digunakan untuk mengamati implementasi nyata arsitektur memori pada sistem operasi Linux dan mengaitkannya dengan teori manajemen memori.

BAB IV

ANALISIS DAN PEMBAHASAN

4.1 ANALISIS TUGAS 1

Berapa jarak (dalam bytes) antara:

- Stack dan Heap?

Alamat Stack (local): 0x7ffd5c3a8a4c

Alamat Heap (heap1): 0x15b9010

Jarak = 0x7ffd5c3a8a4c - 0x15b9010

= 0x7FE7A6CA9A3C bytes

= 140.463.953.740.348 bytes $\approx$ 140 TB

Interprestrasi: Stack dan Heap ditempatkan di ujung berlawanan dari address space virtual, memberikan ruang maksimal bagi keduanya untuk tumbuh (Heap ke atas, Stack ke bawah).

- Text segment dan Data segment?

Alamat Text (func3): 0x400550

Alamat Data (global_var): 0x60104c

Jarak = 0x60104c - 0x400550

= 0x200AFC bytes

= 2.101.500 bytes $\approx$ 2 MB

Interpretasi: Text dan Data segment berdekatan karena merupakan bagian dari executable yang sama. Text segment berisi kode eksekusi, sedangkan Data segment berisi variabel global yang sudah diinisialisasi.

- Alamat fungsi satu dengan lainnya?

func3() ke func1(): 0x400550 - 0x400560 = -0x10 = 16 bytes (backward)

func1() ke func2(): 0x400560 - 0x400570 = -0x10 = 16 bytes (backward)

func2() ke main(): 0x400570 - 0x40057d = -0xD = 13 bytes (backward)

func3() ke main(): 0x400550 - 0x40057d = -0x2D = 45 bytes (backward)

Interpretasi:

- Fungsi-fungsi dalam text segment ditempatkan secara berurutan dalam memori

- Ukuran setiap fungsi berbeda (main = 13 bytes, func1/func2/func3 = 16 bytes)
- Urutan penempatan: func3 → func1 → func2 → main (tergantung urutan deklarasi dan optimasi compiler)

Jalankan program 5 kali, catat alamat-alamat yang muncul:

- Alamat yang SELALU SAMA:
 - Semua alamat di Text Segment (fungsi-fungsi)
 - Semua alamat di Data Segment (variabel global, array statis, string literal)
- Alamat yang BERUBAH:
 - Semua alamat di Heap Segment (alokasi dinamis)
 - Semua alamat di Stack Segment (variabel lokal, parameter fungsi)
- Jelaskan mengapa terjadi perbedaan tersebut (hint: ASLR - Address Space Layout Randomization)

Apa itu ASLR?

ASLR adalah teknik keamanan yang mengacak (randomize) alamat base dari berbagai segment memori setiap kali program dijalankan.

Mekanisme ASLR pada Linux:

- Stack Randomization: Base address stack diacak setiap eksekusi
- Heap Randomization: Base address heap (via brk/mmap) diacak
- Library Randomization: Alamat shared libraries (libc.so, dll) diacak
- Executable Randomization: Dapat diaktifkan via PIE (Position Independent Executable)

Konfigurasi ASLR di Linux:

```
$ cat /proc/sys/kernel/randomize_va_space
2 # 0=disabled, 1=conservative, 2=full randomization
```

Mengapa Text/Data tidak berubah?

Pada program ini, kemungkinan:

- Program dikompilasi tanpa PIE (Position Independent Executable)
- ASLR level 2 biasanya tidak mengacak executable jika bukan PIE
- Text/Data segment memiliki alamat tetap yang ditentukan saat linking

Bukti non-PIE:

Alamat fungsi main() di 0x40057d menunjukkan ini bukan PIE karena:

- PIE executable biasanya mulai dari 0x55... atau 0x56...
- Non-PIE executable biasanya mulai dari 0x400000

Bandingkan dengan memory map dari /proc/[pid]/maps:

- Apakah alamat yang Anda dapatkan sesuai dengan range di memory map?

```
00400000-00401000 r-xp 00000000 08:01 1234567 /home/user/praktikum-
memory/background/address_demo

00600000-00601000 rw-p 00000000 08:01 1234567 /home/user/praktikum-
memory/background/address_demo

015b9000-015da000 rw-p 00000000 00:00 0      [heap]

7ffd5c3a8000-7ffd5c3c9000 rw-p 00000000 00:00 0 [stack]
```

Analisis Mapping:

Segment	Range Memory Map	Permission	Alamat Program	Status
Text	00400000-00401000	r-xp	func3(): 0x400550	Sesuai
Data	00600000-00601000	rw-p	global_var: 0x60104c	Sesuai
Heap	015b9000-015da000	rw-p	heap1: 0x15b9010	Sesuai
Stack	7ffd5c3a8000-7ffd5c3c9000	rw-p	local: 0x7ffd5c3a8a4c	Sesuai

- Sebutkan permission (rwx) dari setiap segment

Segment	Permission	Keterangan
Text	r-xp	Read + Execute (kode program)
Data	rw-p	Read + Write (global & static)
Heap	rw-p	Read + Write (malloc)
Stack	rw-p	Read + Write (local variable)

4.2 ANALISIS TUGAS 2

Algoritma	Waktu Eksekusi (μs)	Fragmentasi (%)	Failed Allocations
First-Fit	115	00.00	1
Best-Fit	170	00.00	0

Worst-Fit	200	00.00	2
Next-Fit	105	00.00	1

Next-Fit merupakan algoritma paling cepat berdasarkan hasil pengujian.

Hal ini karena Next-Fit tidak memulai pencarian blok kosong dari awal memori, melainkan melanjutkan pencarian dari posisi alokasi terakhir. Dengan demikian, waktu pencarian menjadi lebih singkat dan overhead pencarian dapat dikurangi.

Algoritma mana yang paling efisien dalam mengurangi fragmentasi?

Best-Fit merupakan algoritma paling efisien dalam mengurangi fragmentasi.

Algoritma ini memilih blok kosong dengan ukuran yang paling mendekati kebutuhan proses, sehingga sisa ruang yang tidak terpakai menjadi lebih kecil dan fragmentasi eksternal dapat diminimalkan.

Dalam situasi apa setiap algoritma lebih unggul?

- **First-Fit**
Cocok digunakan pada sistem yang membutuhkan implementasi sederhana dan cepat tanpa mempertimbangkan fragmentasi secara mendalam.
- **Best-Fit**
Lebih unggul pada sistem dengan keterbatasan memori, karena mampu memanfaatkan ruang memori secara lebih efisien.
- **Worst-Fit**
Sesuai untuk skenario tertentu yang membutuhkan blok memori besar di masa depan, meskipun berpotensi menghasilkan fragmentasi yang lebih tinggi.
- **Next-Fit**
Sangat efektif pada sistem dengan banyak permintaan alokasi berurutan, karena mengurangi waktu pencarian dan meningkatkan kecepatan eksekusi.

4.3 ANALISIS TUGAS 3

caranya menjalankan menggunakan referensestring yang sama

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1

total hasilnya = 20

Tabel perbandingan FIFO VS LRU

Algoritma	Page Fault	Hit Ratio
FIFO	15	25%
LRU	12	40%

Buat tabel trace untuk setiap algoritma

Tabel Trace FIFO (4 Frames)

Step	Ref	F0	F1	F2	F3	Fault
1	7	7	-	-	-	Y
2	0	7	0	-	-	Y
3	1	7	0	1	-	Y
4	2	7	0	1	2	Y
5	0	7	0	1	2	N
6	3	3	0	1	2	Y
7	0	3	0	1	2	N
8	4	3	4	1	2	Y
9	2	3	4	1	2	N
10	3	3	4	1	2	N
11	0	3	4	0	2	Y
12	3	3	4	0	2	N
13	2	3	4	0	2	N
14	1	1	4	0	2	Y
15	2	1	4	0	2	N
16	0	1	4	0	2	N
17	1	1	4	0	2	N
18	7	1	7	0	2	Y
19	0	1	7	0	2	N
20	1	1	7	0	2	N

FIFO – Perhitungan

- Page Fault = **15**
- Page Hit = $20 - 15 = 5$
- Hit Ratio = $5 / 20 = \mathbf{25\%}$
- Disk Writes = **0** (tidak ada write)

Tabel Trace LRU (4 Frames)

Step	Ref	F0	F1	F2	F3	Fault
1	7	7	-	-	-	Y
2	0	7	0	-	-	Y
3	1	7	0	1	-	Y
4	2	7	0	1	2	Y
5	0	7	0	1	2	N
6	3	3	0	1	2	Y
7	0	3	0	1	2	N
8	4	3	0	4	2	Y
9	2	3	0	4	2	N
10	3	3	0	4	2	N
11	0	3	0	4	2	N
12	3	3	0	4	2	N
13	2	3	0	4	2	N
14	1	3	0	1	2	Y
15	2	3	0	1	2	N
16	0	3	0	1	2	N
17	1	3	0	1	2	N
18	7	7	0	1	2	Y
19	0	7	0	1	2	N
20	1	7	0	1	2	N

LRU – Perhitungan

- Page Fault = **12**
- Page Hit = $20 - 12 = 8$
- Hit Ratio = $8 / 20 = 40\%$
- Disk Writes = **0**
- **Jalankan dengan 3, 4, dan 5 frames**
- caranya menjalankan menggunakan referensi string yang sama
- 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1
total hasilnya = 20

Hasil eksekusi

➤ FIFO

Jumlah Frame	Page Fault	Page Hit	Hit Ratio
3 frames	16	4	20%
4 frames	15	5	25%
5 frames	16	4	20%

➤ LRU

Jumlah Frame	Page Fault	Page Hit	Hit Ratio
3 frames	18	2	10%
4 frames	12	8	40%
5 frames	10	10	50%

Apakah page fault berkurang seiring bertambahnya frame?

- **FIFO → TIDAK SELALU**
 - Terjadi Belady's Anomaly
- **LRU → YA**
 - Page fault selalu berkurang

Analisis performa

- **Algoritma mana lebih baik?**

LRU lebih baik karena mempertahankan halaman yang paling sering digunakan sehingga menghasilkan page fault lebih sedikit.

- Kapan FIFO lebih baik?

FIFO lebih sederhana dan cocok untuk sistem kecil dengan keterbatasan resource, meskipun kurang optimal.

4.4 ANALISIS TUGAS 4

Struktur Page Table	Jumlah Entri	Ukuran per Entri	Total Kebutuhan Memori	Efisiensi Ruang
Single-Level	2^{44}	4–8 byte	Sangat besar (tidak praktis)	Rendah
Two-Level	Dinamis	4–8 byte	Lebih kecil dari single-level	Sedang
Three-Level	Sangat dinamis	4–8 byte	Jauh lebih efisien	Tinggi
Inverted	#NAME?	±12 byte	Paling kecil	Sangat tinggi

1. Jumlah Memory Access untuk Setiap Struktur Page Table

Jumlah memory access yang dibutuhkan untuk menerjemahkan alamat virtual ke alamat fisik bergantung pada struktur page table yang digunakan.

Struktur Page Table	Memory Access (tanpa TLB)	Penjelasan
Single-Level	1 kali	Page table diakses satu kali untuk mendapatkan frame number.
Two-Level	2 kali	Akses pertama ke page table level 1, akses kedua ke page table level 2.

Three-Level	3 kali	Akses berurutan ke level 1, level 2, dan level 3.
Inverted	Banyak ($O(n)$)	Pencarian pasangan (<i>PID, page number</i>) di seluruh tabel atau melalui hashing.

2. Pengaruh TLB (Translation Lookaside Buffer)

TLB adalah cache berkecepatan tinggi yang menyimpan hasil translasi alamat virtual ke alamat fisik.

a. Saat Terjadi TLB Hit

Jika alamat virtual ditemukan di TLB:

- Semua struktur page table hanya membutuhkan 1 memory access
- Page table tidak perlu diakses sama sekali
- Performa sistem meningkat secara signifikan

Struktur	Memory Access saat TLB Hit
Single-Level	1
Two-Level	1
Three-Level	1
Inverted	1

b. Saat Terjadi TLB Miss

Jika alamat virtual **tidak ada** di TLB:

- Sistem harus mengakses page table
- Jumlah akses mengikuti struktur page table

Struktur	Memory Access saat TLB Miss
Single-Level	1
Two-Level	2
Three-Level	3
Inverted	Banyak (linear / hash)

1. Struktur Page Table yang Cocok untuk Sistem 32-bit dan 64-bit

a. Sistem 32-bit

Pada sistem 32-bit, ruang alamat virtual relatif kecil (maksimum 4 GB). Oleh karena itu:

- Two-level page table merupakan pilihan yang paling sesuai
- Memory overhead masih dapat dikendalikan
- Kecepatan translasi relatif cepat
- Implementasi lebih sederhana dibanding three-level

2. Kapan Inverted Page Table Lebih Efisien?

Inverted page table lebih efisien pada kondisi berikut:

1. **Ruang alamat virtual sangat besar**
 Karena ukuran tabel bergantung pada **jumlah frame fisik**, bukan jumlah page virtual.
2. **Jumlah frame fisik terbatas**
 Semakin sedikit frame, semakin kecil ukuran inverted page table.
3. **Sistem multi-proses**
 Inverted page table menyimpan pasangan (*process ID, page number*) sehingga cocok untuk banyak proses.

4. Didukung oleh TLB atau hashing

Tanpa TLB, pencarian inverted page table lambat. Dengan TLB atau hash table, performanya meningkat signifikan.

4.5 ANALISIS TUGAS 5

Strategy	Swap Out Count	Swap In Count	Total Swap Time	AVG Respons Time
FIFO	33	26	8519 ms	85.19 ms
LRU	39	33	10152 ms	101.52 ms
Priority	37	30	8830 ms	88.30 ms

- **Strategi mana yang meminimalkan thrashing?**

Berdasarkan data hasil simulasi, strategi **FIFO** paling efektif dalam meminimalkan thrashing. Hal ini terlihat dari jumlah swap in/out serta total swap time yang paling rendah dibandingkan LRU dan Priority. Nilai average response time FIFO juga paling kecil, menunjukkan sistem tidak terlalu sering melakukan pertukaran proses secara berulang, sehingga aktivitas thrashing dapat ditekan. LRU justru mengalami thrashing lebih tinggi karena sering melakukan penggantian proses akibat pola akses acak, sedangkan Priority berada di tingkat menengah.

- **Bagaimana ukuran memory mempengaruhi performa?**

Ukuran memori sangat memengaruhi performa sistem. Semakin kecil memori fisik dibandingkan total kebutuhan proses, semakin sering terjadi swapping, yang meningkatkan swap time dan menurunkan waktu respons. Sebaliknya, penambahan memori fisik dapat mengurangi frekuensi swapping, menekan thrashing, dan meningkatkan stabilitas sistem.

- **Kapan swapping lebih baik daripada killing process?**

Swapping lebih baik daripada killing process ketika proses masih memiliki kemungkinan untuk dilanjutkan, misalnya pada sistem time-sharing atau multi-user. Killing process hanya layak dilakukan jika sistem berada dalam kondisi kritis atau proses tidak penting, karena swapping memungkinkan sistem tetap responsif tanpa kehilangan pekerjaan yang sedang berjalan.

4.6 ANALISIS TUGAS 6

Bandingkan spesifikasi sistem Anda dengan:

- Intel 32-bit (IA-32) architecture
- Intel 64-bit (x86-64) architecture
- ARMv8 architecture (jika tersedia)

Berdasarkan perintah lscpu, sistem yang digunakan memiliki spesifikasi berikut:

- Architecture: x86_64
- CPU: 11th Gen Intel Core i3-1115G4
- Mode operasi CPU: mendukung 32-bit dan 64-bit
- Jumlah core: 2 core
- Thread: 2 thread per core (total 4 thread)

Dengan demikian, sistem ini menggunakan arsitektur Intel 64-bit (x86-64).

Perbandingan dengan arsitektur lain:

- Intel IA-32 (32-bit): Mendukung address space lebih kecil (32-bit), saat ini tidak digunakan sebagai mode utama
- Intel x86-64 (64-bit): Digunakan pada sistem ini, mendukung address space besar dan multi-level paging modern
- ARMv8: Tidak relevan karena sistem menggunakan prosesor Intel, bukan ARM

Berapa bit address space yang digunakan?

- Arsitektur CPU: 64-bit
- Namun, pada implementasi x86-64 modern:
 - Virtual address tidak menggunakan penuh 64 bit
 - Umumnya menggunakan 48-bit virtual address

Perhitungan Address Space:

2^{48} byte = 256 TB

Bagaimana implementasi multi-level paging?

a. Ukuran Page

getconf PAGE_SIZE

Hasil:

4096 bytes (4 KB)

b. Struktur Multi-Level Paging (x86-64)

Pada arsitektur x86-64, paging diimplementasikan dengan 4-level paging, yaitu:

Level	Nama
Level 1	PML4 (Page Map Level 4)
Level 2	PDPT (Page Directory Pointer Table)
Level 3	Page Directory
Level 4	Page Table

Setiap level menggunakan 9 bit index, dan:

- Offset page: 12 bit
- Total: $9 + 9 + 9 + 9 + 12 = 48$ bit

c. Dukungan TLB (Translation Lookaside Buffer)

Output `cpuid | grep -i tlb` menunjukkan:

- TLB untuk 4 KB pages
- TLB untuk 2 MB / 4 MB pages
- L1 dan L2 TLB tersedia

Ini menandakan:

- Sistem mendukung multiple page sizes
- Optimalisasi translasi alamat melalui TLB caching

Berapa ukuran maksimum virtual memory?

Secara teori (x86-64):

- Maksimum virtual memory: 256 TB per proses

Secara praktik pada sistem ini:

- RAM fisik: 3.7 GB
- Swap: 1 GB

Virtual memory yang bisa dialokasikan jauh lebih besar dari RAM fisik

q

BAB V

KESIMPULAN DAN SARAN

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa manajemen memori merupakan komponen krusial dalam sistem operasi yang sangat memengaruhi kinerja dan stabilitas sistem. Melalui implementasi berbagai simulasi seperti memory layout, allocator, paging, page replacement, dan swapping, mahasiswa dapat memahami bagaimana sistem operasi mengelola memori secara efisien di tengah keterbatasan sumber daya. Perbandingan algoritma menunjukkan bahwa setiap metode memiliki kelebihan dan kekurangan tergantung pada kondisi beban kerja dan pola akses memori. Selain itu, pengamatan langsung terhadap arsitektur sistem Linux memperkuat pemahaman bahwa konsep teoretis benar-benar diterapkan pada sistem nyata. Praktikum ini memberikan pengalaman teknis dan analitis yang penting sebagai bekal dalam memahami sistem operasi modern secara mendalam.

DAFTAR PUSTAKA

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10th ed.). Wiley.